Using AI Coding Assistants in the Lab

Benjamin-Chung Lab Manual

Updated: 2026-07-04

Contents

1 Welcome to the Benjamin-Chung Lab! 7
- 1.1 About the lab . . . 7
- 1.2 About this lab manual . . . 7

2 Culture and conduct 9
- 2.1 Lab culture . . . 9
- 2.2 Diversity, equity, and inclusion . . . 9
- 2.3 Protecting human subjects . . . 9
- 2.4 Authorship . . . 10

3 Communication and coordination 11
- 3.1 Slack . . . 11
- 3.2 Email . . . 11
- 3.3 Trello . . . 12
- 3.4 Google Drive . . . 12
- 3.5 Stanford Medicine Box . . . 12
- 3.6 Meetings . . . 12

4 Reproducibility 13
- 4.1 What is the reproducibility crisis? . . . 13
- 4.2 Study design . . . 15
- 4.3 Register study protocols . . . 15
- 4.4 Write pre-analysis plans . . . 15
- 4.5 Register pre-analysis plans . . . 15
- 4.6 Create reproducible workflows . . . 16
- 4.7 Process and analyze data with internal replication and masking . 17
- 4.8 Use reporting checklists with manuscripts . . . 17
- 4.9 Publish preprints . . . 17
- 4.10 Publish data (when possible) and replication scripts . . . 17

5 Getting started 19
- 5.1 The tools and how they connect . . . 19
- 5.2 RStudio . . . 21

5.3 GitHub intro . . . 21
5.4 The terminal . . . 22
5.5 Sherlock . . . 22
5.6 Box . . . 22
5.7 A typical first week . . . 23

6 Using agentic AI coding assistants 25
6.1 Three key issues . . . 26
6.2 Bottom line . . . 27
6.3 Suggested safeguards . . . 30
6.4 Know your target journal's AI policy before you start . . . 32

7 Code repositories 35
7.1 Project Structure . . . 35
7.2 `0-functions` folder . . . 36
7.3 `.Rproj` files . . . 36
7.4 Configuration ('config') File . . . 37
7.5 Order Files and Directories . . . 38
7.6 Using Bash scripts to ensure reproducibility . . . 38

8 Coding practices 39
8.1 Organizing scripts . . . 39
8.2 Documenting your code . . . 39
8.3 Object naming . . . 41
8.4 Function calls . . . 42
8.5 The here package . . . 43
8.6 Reading/Saving Data . . . 43
8.7 Tidyverse . . . 43
8.8 Coding with R and Python . . . 45
8.9 Repeating analyses with different variations . . . 46
8.10 Reviewing Code . . . 49

9 Coding style 53
9.1 Comments . . . 53
9.2 Line breaks . . . 54
9.3 Automated Tools for Style and Project Workflow . . . 56

10 Working with Big Data 59
10.1 The data.table package . . . 59
10.2 Using downsampled data . . . 60
10.3 Optimal RStudio set up . . . 60

11 Tidy evaluation and programming with dplyr 61
11.1 General Overview . . . 61
11.2 Technical Overview . . . 63

12 Data validation and unit testing 65

12.1 Data validation with assertions in analysis scripts 65
12.2 Writing a validation function 69
12.3 Unit testing with `testthat` 70
12.4 Summary of practices 74

13 Github **75**
13.1 Basics 75
13.2 Github Desktop 75
13.3 Git Branching 75
13.4 Example Workflow 76
13.5 Commonly Used Git Commands 77
13.6 How often should I commit? 78
13.7 What should be pushed to Github? 78

14 Unix commands **79**
14.1 Basics 79
14.2 Syntax for both Mac/Windows 79
14.3 Running Bash Scripts 80
14.4 Running Rscripts in Windows 80
14.5 Checking tasks and killing jobs 81
14.6 Running big jobs 83

15 Reproducible Environments **85**
15.1 Package Version Control with renv 85
15.2 Documenting the R version 87
15.3 Ensuring consistency between local and cluster environments . . 87

16 Code Publication **89**
16.1 Checklist overview 89
16.2 Fill out file headers 89
16.3 Clean up comments 89
16.4 Document functions 90
16.5 Remove deprecated filepaths 90
16.6 Ensure project runs via bash 90
16.7 Complete the README 90
16.8 Clean up feature branches 92
16.9 Create Github release 92

17 Data Publication **93**
17.1 Overview 93
17.2 Removing PHI 94
17.3 Create public IDs 95
17.4 Create a data repository 97
17.5 Edit and test analysis scripts 97
17.6 Create a public GitHub page for public scripts 97
17.7 Go live 98

18 Slurm and cluster computing **99**
18.1 Getting started . . . 99
18.2 Moving files to Sherlock . . . 100
18.3 Installing packages on Sherlock . . . 100
18.4 Testing your code . . . 101
18.5 Storage & group storage access . . . 102
18.6 Running big jobs . . . 103

19 Checklists **105**
19.1 Pre-analysis plan checklist . . . 105
19.2 Code checklist . . . 105
19.3 Manuscript checklist . . . 106
19.4 Figure checklist . . . 107

20 Resources **109**
20.1 Resources for R . . . 109
20.2 Resources for Git & Github . . . 109
20.3 Scientific figures . . . 109
20.4 Writing . . . 110
20.5 Presentations . . . 110
20.6 Professional advice . . . 110
20.7 Funding . . . 110
20.8 Ethics and global health research . . . 110

Chapter 1

Welcome to the Benjamin-Chung Lab!

1.1 About the lab

Welcome to the lab of Dr. Jade Benjamin-Chung, Assistant Professor in Epidemiology & Population Health at Stanford University. Our mission is to improve population health by creating high quality evidence about what health interventions work in whom and where, when, and how to implement them. Most of our research is focused on infectious diseases, including malaria, diarrhea, soil-transmitted helminths, and influenza. Our focus is on improving the health of vulnerable populations from low-resource settings, both domestically and internationally. We use a variety of epidemiologic, computational, and statistical methods, including causal inference and machine learning methods, in pursuit of our mission. To learn more about the lab, visit jadebc.net.

1.2 About this lab manual

This lab manual covers our communication strategy, code of conduct, and best practices for reproducibility of computational workflows. It is a living document that is updated regularly.

Most of the code examples in this manual are written in R, which is the primary language used in our lab. However, the core principles — reproducibility, documentation, version control, data validation, and code review — apply to any language. Where a practice is R-specific (e.g., `renv`, tidyverse conventions), we note it as such.

This manual was created with input from a large number of team members and with inspiration from other scientists' lab manuals. Contributors include Jade

Benjamin-Chung, Kunal Mishra, Stephanie Djajadi, Nolan Pokpongkiat, Anna Nguyen, Iris Tong, and Gabby Barratt Heitmann. Some chapters were drafted with AI assistance from Claude (Anthropic).

Feel free to draw from this manual (and please cite it if you do!).

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License.

Chapter 2

Culture and conduct

by Jade Benjamin-Chung

2.1 Lab culture

We are committed to a lab culture that is collaborative, supportive, inclusive, open, and free from discrimination and harassment.

We encourage students / staff of all experience levels to respectfully share their honest opinions and ideas on any topic. Our group has thrived upon such respectful honest input from team members over the years, and this document is a product of years of student and staff input (and even debate) that has gradually improved our productivity and overall quality of our work.

2.2 Diversity, equity, and inclusion

The Benjamin-Chung lab recognizes the importance of and is committed to cultivating a culture of diversity, equity, and inclusion. This means being a safe, supportive, and anti-racist environment in which students from diverse backgrounds are equally and inclusively supported in their education and training. Diversity takes many forms, and includes, but is not limited to, differences in race, ethnicity, gender, sexuality, socioeconomic status, religion, disability, and political affiliation.

2.3 Protecting human subjects

All lab members must complete CITI Human Subjects Biomedical Group 1 training and share their certificate with Jade. She will add team members to

relevant Institutional Review Board protocols prior to their start date to ensure they have permission to work with identifiable datasets.

One of the most relevant aspects of protecting human subjects in our work is maintaining confidentiality. For students supporting our data science efforts, in practice this means:

- Be sure to understand and comply with project-specific policies about where data can be saved, particularly if the data include personal identifiers.
- Do not share data with anyone without permission, including to other members of the group, who might not be on the same IRB protocol as you (check with Jade first).

Remember, data that looks like it does not contain identifiers to you might still be classified as data that requires special protection by our IRB or under HIPAA, so always proceed with caution and ask for help if you have any concerns about how to maintain study participant confidentiality.

2.4 Authorship

We adhere to the ICMJE Definition of authorship and are happy for team members who meet the definition of authorship to be included as co-authors on scientific manuscripts.

Chapter 3

Communication and coordination

by Jade Benjamin-Chung

One benefit of the academic environment is its schedule flexibility. This means that lab members may choose to work in the early morning, evening, or weekends. That said, we do not expect lab members to respond outside of business hours (unless there are special circumstances).

3.1 Slack

- Use Slack for scheduling, coding related questions, quick check ins, etc. If your Slack message exceeds 200 words, it might be time to use email.
- Use channels instead of direct messages unless you need to discuss something private.
- Please make an effort to respond to messages that message you (e.g., `@jade`) as quickly as possible and always within 24 hours.
- If you are unusually busy (e.g., taking MCAT/GRE, taking many exams) or on vacation please alert the team in advance so we can expect you not to respond at all / as quickly as usual and also set your status in Slack (e.g., it could say "On vacation") so we know not to expect to see you online.
- Please thread messages in Slack as much as possible.

3.2 Email

- Use email for longer messages (>200 words) or messages that merit preservation.

- Generally, strive to respond within 24 hours hours. As noted above, if you are unusually busy or on vacation please alert the team in advance so we can expect you not to respond at all / as quickly as usual.

3.3 Trello

- Jade will add new cards within our shared Trello board that outline your tasks.
- The higher a card is within your list, the higher priority it is.
- Generally, strive to complete the tasks in your card by the date listed.
- Use checklists to break down a task into smaller chunks. Sometimes Jade will write this for you, but you can also add this yourself.
- Jade will move your card to the "Completed" list when it is done.

3.4 Google Drive

- We mostly use Google Drive to create shared documents with longer descriptions of tasks. These documents are linked to in Trello. Jade often shares these with the whole team since tasks are overlapping, and even if a task is assigned to one person, others may have valuable insights.

3.5 Stanford Medicine Box

- Human subjects data for research studies are generally stored in Stanford Medicine Box. Please check with Jade about whether there are special storage and transfer requirements for the datasets you are working with for each study.
- If you are in the School of Medicine, you can access Box via your SUNet ID. If you are no in the School of Medicine, please follow these instructions to get a Box account.

3.6 Meetings

- Our meetings start on the hour.
- If you are going to be late, please send a message in our Slack channel.
- If you are regularly not able to come on the hour, notify the team and we might choose the modify the agenda order or the start time.

Chapter 4

Reproducibility

by Jade Benjamin-Chung

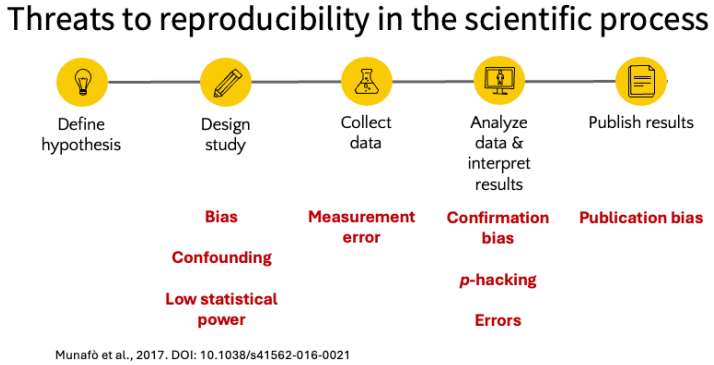
Our lab adopts the following practices to maximize the reproducibility of our work.

1. Design studies with appropriate methodology and adherence to best practices in epidemiology and biostatistics
2. Register study protocols
3. Write and register pre-analysis plans
4. Create reproducible workflows
5. Process and analyze data with internal replication and masking
6. Use reporting checklists with manuscripts
7. Publish preprints
8. Publish data (when possible) and replication scripts

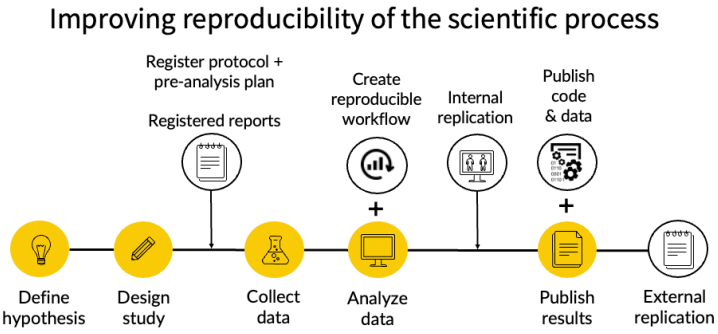
4.1 What is the reproducibility crisis?

In the past decade, an increasing number of studies have found that published study findings could not be reproduced. Researchers found that it was not possible to reproduce estimates from published studies: 1) with the same data and same or similar code and 2) with newly collected data using the same (or similar) study design. These "failures" of reproducibility were frequent enough and broad enough in scope, occurring across a range of disciplines (epidemiology, psychology, economics, and others) to be deeply troubling. Program and policy decisions based on erroneous research findings could lead to wasted resources, and at worst, could harm intended beneficiaries. This crisis has motivated new practices in reproducibility, transparency, and openness. Our lab is committed to adopting these best practices, and much of the remainder of the lab manual focuses on how to do so.

The scientific process involves a series of steps from defining a hypothesis through publishing results, and threats to reproducibility can arise at each stage. The figure below summarizes common threats that can occur at each step in the scientific process, including bias and low statistical power in study design, measurement error during data collection, and confirmation bias, p-hacking, and errors during analysis.



Our lab attempts to minimize these threats through a set of practices at each stage of the research process. The figure below illustrates our approach: we register protocols and pre-analysis plans before data collection, create reproducible computational workflows and conduct internal replication during analysis, and publish code and data alongside our results to enable external replication. The rest of this chapter describes each of these practices in detail.



Recommended readings on the "reproducibility crisis":

- Nuzzo R. How scientists fool themselves – and how they can stop. 2015. https://www.nature.com/articles/526182a
- Stoddart C. Is there a reproducibility crisis in science? 2016. https://www.nature.com/articles/d41586-019-00067-3
- Munafo MR, et al. A manifesto for reproducible science. *Nature Human Behavior* 2017 http://dx.doi.org/10.1038/s41562-016-0021

4.2 Study design

Appropriate study design is beyond the scope of this lab manual and is something trainees develop through their coursework and mentoring.

4.3 Register study protocols

We register all randomized trials on clinicaltrials.gov, and in some cases register observational studies as well.

4.4 Write pre-analysis plans

We write pre-analysis plans for most original research projects that are not exploratory in nature, although in some cases, we write pre-analysis plans for exploratory studies as well. The format and content of pre-analysis plans can vary from project to project. Generally, these include:

1. Brief background on the study (a condensed version of the introduction section of the paper)
2. Hypotheses / objectives
3. Study design
4. Description of data
5. Definition of outcomes
6. Definition of interventions / exposures
7. Definition of covariates
8. Statistical power calculation
9. Statistical analysis:

- Type of model
- Covariate selection / screening
- Standard error estimation method
- Missing data analysis
- Assessment of effect modification / subgroup analyses
- Sensitivity analyses
- Negative control analyses

See the following pre-analysis plans for examples from our lab:

- Trials of water, sanitation, and hygiene interventions
- Observational study of school-located influenza vaccination
- Complex re-analysis of malaria intervention trial

4.5 Register pre-analysis plans

We use Open Science Framework to register pre-analysis plans. Learn more here. Briefly, what this entails is creating a time-stamped, publicly accessible record

of your hypotheses, study design, and analysis plan before you analyze the data. Registration does not prevent you from conducting additional analyses — it simply makes transparent which analyses were planned in advance and which were exploratory. To register, create a project on OSF, upload your pre-analysis plan document, and select "Register" to create a frozen, time-stamped version. Once registered, the plan cannot be edited or deleted, though you can add addenda if your analysis plan changes after registration. You can also include a link to the registration at the time of submitting your manuscript. This shows peer reviewers and editors (and later, readers) that you pre-specified your analysis.

People often ask, well what if I need to change my plan after pre-specifying it? No problem! We commonly include a list of deviations from the pre-analysis plan in the supplement of our manuscript. In our experience, this has only strengthened reviewer and editor perceptions of the reproducibility and rigor of our work because it shows that we are transparent about our process and that we are not hiding any changes to the analysis plan. We have even included a list of deviations resulting from the peer review process as its own section in the supplement to make those changes transparent as well.

4.6 Create reproducible workflows

Reproducible workflows allow a user to reproduce study estimates and ideally figures and tables with a "single click". In practice, this typically means running a single bash script that sources all replication scripts in a repository. These replication scripts complete data processing, data analysis, and figure/table generation. The following chapters provide detailed guidance on this topic:

- Getting started: Overview of the tools used in our lab
- Code repositories: Project structure and configuration
- Coding practices: Script organization, documentation, and code review
- Coding style: Formatting conventions
- Data validation and unit testing: Assertions, sanity checks, and unit testing
- Working with big data: Handling large datasets
- Tidy evaluation and programming with dplyr: Programming with tidyverse functions
- GitHub: Version control and collaboration
- Unix commands: Running scripts from the command line
- Reproducible environments: Package and R version management
- Code publication: Preparing code for public release

4.7 Process and analyze data with internal replication and masking

See my video on this topic: https://www.youtube.com/watch?v=WoYkY9MkbRE

4.8 Use reporting checklists with manuscripts

Using reporting checklists helps ensure that peer-reviewed articles contain the information needed for readers to assess the validity of your work and/or attempt to reproduce it. A collection of reporting checklists is available here: https://www.equator-network.org/about-us/what-is-a-reporting-guideline/)

4.9 Publish preprints

A preprint is a scientific manuscript that has not been peer reviewed. Preprint servers create digital object identifiers (DOIs) and can be cited in other articles and in grant applications. Because the peer review process can take many months, publishing preprints prior to or during peer review enables other scientists to immediately learn from and build on your work. Importantly, NIH allows applicants to include preprint citations in their biosketches. In most cases, we publish preprints on medRxiv.

4.10 Publish data (when possible) and replication scripts

Publishing data and replication scripts allows other scientists to reproduce your work and to build upon it. We typically publish data on Open Science Framework, share links to Github repositories, and archive code on Zenodo.

Chapter 5

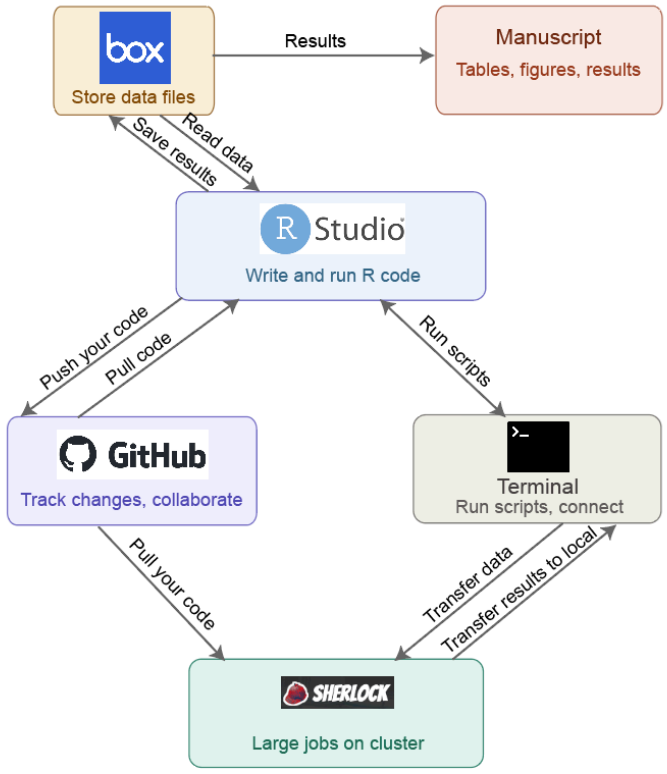
Getting started

by Jade Benjamin-Chung

This chapter is for students who are new to computational research. It provides a brief overview of the core tools you will use in our lab and how they fit together. Each tool is covered in more detail in later chapters — the goal here is to give you the big picture before you dive in.

5.1 The tools and how they connect

A typical project in our lab involves writing R code to clean data, run statistical analyses, and produce tables and figures for a manuscript. That code lives in a GitHub repository so that multiple people can collaborate on it and so that every change is tracked. Some projects involve large datasets or computationally intensive analyses that need to run on Stanford's Sherlock computing cluster rather than your laptop. The terminal (command line) is how you interact with Sherlock and how you run scripts outside of RStudio.



How the tools in our lab typically connect
Most of your time is spent in RStudio; the other tools support the workflow around it

Here is how these tools relate to each other:

- **RStudio** is where you write and run R code interactively. You will spend most of your time here.
- **GitHub** is where your code is stored and shared. It tracks every change, lets multiple people work on the same project, and is where code review happens.
- **The terminal** (also called the command line) is how you run scripts outside of RStudio, interact with GitHub from the command line, connect to Sherlock, and run bash scripts that execute your full analysis pipeline.
- **Sherlock** is Stanford's computing cluster. You use it when your analysis is too slow or too memory-intensive to run on your laptop.
- **Box** is where we store large data files that should not be pushed to GitHub (e.g., datasets with protected health information).

You do not need to be an expert in all of these before starting. Most students learn them incrementally as they work on their first project. But you should have each tool set up and be comfortable with the basics described below before

you begin coding.

5.2 RStudio

RStudio is an integrated development environment (IDE) for R. It provides a text editor for writing scripts, a console for running code interactively, a file browser, and tools for viewing plots and data. Download it from posit.co.

A few things to set up right away:

- **Turn off saving the workspace on exit.** Go to Tools > Global Options > General and set "Save workspace to .RData on exit" to "Never." This prevents old objects from persisting between sessions, which is a common source of irreproducible results.
- **Enable soft-wrap.** Go to Tools > Global Options > Code and check "Soft-wrap R source files" so that long lines wrap on screen rather than running off the edge.
- **Use RStudio projects.** Every project in our lab should have an `.Rproj` file. When you open an `.Rproj` file, RStudio sets the working directory to the project folder, which means file paths work consistently for everyone. See the Code repositories chapter for more detail.

If you are new to R, work through the first few chapters of R for Data Science before starting your first project. If you took an introductory R course, you likely already know enough to get started — most of what you need beyond that you will learn on the job.

5.3 GitHub intro

GitHub is a platform for version control and collaboration. In practice, this means:

- Every change you make to your code is recorded with a short description (a "commit"). If something breaks, you can see exactly what changed and revert it.
- Multiple people can work on the same codebase without overwriting each other's work.
- Before code is incorporated into the main project, it goes through a code review via a "pull request," where another team member checks it for correctness and clarity.

To get started:

1. Create a free account at github.com.
2. Install GitHub Desktop, which provides a graphical interface for common Git operations (cloning, committing, pushing, pulling). You can also use

Git from the terminal, but GitHub Desktop is easier when you are starting out.
3. Ask Jade to add you to the relevant Github repositories.

You do not need to understand Git deeply to start — the basics are: **pull** (download the latest code), **commit** (save a set of changes with a description), and **push** (upload your commits to GitHub). The GitHub chapter covers this in more detail, including branching and pull requests.

5.4 The terminal

The terminal (called Terminal on Mac, or Git Bash on Windows) is a text-based interface for interacting with your computer. You will use it to:

- Run bash scripts that execute your full analysis pipeline end-to-end
- Connect to Sherlock via SSH
- Use Git from the command line (optional if you prefer GitHub Desktop)

If you have never used the terminal before, the key concepts are:

- You type commands and press Enter to execute them.
- `cd` changes your directory (folder), `ls` lists the files in your current directory, and `pwd` shows where you are.
- You run an R script from the terminal with `Rscript my_script.R` or a bash script with `bash run_all.sh`.

On Mac, Terminal is pre-installed (find it in Applications > Utilities). On Windows, install Git for Windows, which includes Git Bash. The Unix commands chapter covers the commands you will use most often.

5.5 Sherlock

Sherlock is Stanford's high-performance computing cluster. You will use it when your analysis requires more memory or computing power than your laptop can provide — for example, running bootstrapped confidence intervals across many subgroups, or fitting models on large datasets.

You do not need Sherlock for every project, and you will typically develop and test your code locally in RStudio first. When you are ready to run on Sherlock, the Slurm and cluster computing chapter walks through the setup. To get access, ask Jade to request an account for you.

5.6 Box

We use Stanford Medicine Box to store data files that are too large for GitHub or that contain protected health information (PHI). GitHub repositories should

never contain data — they contain only code, documentation, and small configuration files. Your project's configuration file (`0-config.R`) will include paths that point to the relevant Box folder so that your scripts can read and write data. See the Communication and coordination chapter for more on how we use Box.

5.7 A typical first week

When you join the lab and are assigned to a project, here is roughly what your first week of setup looks like:

1. Install R, RStudio, and GitHub Desktop. Configure RStudio as described above.
2. Create a GitHub account and get added to the lab organization.
3. Clone the project repository using GitHub Desktop.
4. Open the `.Rproj` file in RStudio.
5. Run `renv::restore()` to install the correct package versions (see Reproducible environments).
6. Get access to the project's Box folder and set up your local Box sync so that data paths in the config file work.
7. Try running the project's main bash script or a single analysis script to make sure everything is set up correctly.

If something doesn't work, that is normal — environment setup is often the hardest part. Ask a current lab member for help before spending hours debugging on your own.

Chapter 6

Using agentic AI coding assistants

Tools like Claude Code (and comparable agentic coding tools from other vendors) are different from asking a chatbot to help you write a snippet of code. This distinction matters for how we use these tools in research:

- **Chat-based help** (e.g., asking a chatbot to explain a function or debug a pasted error) keeps you in the loop for every step. You decide what code to run, when, and where.
- **Agentic coding tools** run directly in your terminal or code editor (e.g., RStudio), can read and edit files across your project, execute code, run tests, and iterate on its own output — all with a degree of autonomy that you grant it. It can plan a multi-step task, write and execute code across several files, install packages, and run analyses with minimal intervention. This is powerful, but it also means more can happen without your direct review at each step. That's why coding with these tools is called "vibe coding". Agentic tools include Claude Code, Codex, Gemini CLI, Cursor, and Github Co-Pilot.

Both modes are useful. This chapter includes our Lab Policy on use of AI in Research and then discusses the rationale in detail with a focus on the agentic mode, since it raises issues that are new for our research workflows.

6.0.1 Benjamin-Chung Lab Policy on use of AI in Research

- **Lab members must use Stanford-licensed enterprise AI tools** (Claude for Education, ChatGPT Edu, Gemini Enterprise) for all research-related work involving lab data, code, or drafts. These tools carry Stan-

ford's data protection and institutional agreements; personal/free-tier accounts do not.

- **Tool choice must match the data risk level and requirements in the IRB protocol and Data Use Agreement, if applicable.**
  - **Low risk** (e.g., public data, non-sensitive code, general drafts): any Stanford-licensed enterprise tool may be used.
  - **Moderate risk** (e.g., de-identified data, unpublished results, internal code, grant/manuscript drafts): Stanford-licensed enterprise tools only — no personal or free-tier accounts.
  - **High risk / PHI** (e.g., identifiable or participant-level data, protected health information): only tools explicitly approved for this risk tier may be used, and only after confirming with Jade that the specific tool and use case are permitted under the project's IRB protocol and data use agreement. Some Stanford-licensed tools may not be approved for PHI use even if generally approved for lab work — check before proceeding.
- **No lab data, code, or manuscripts can be used in non-approved tools.** This includes participant data (even de-identified), unpublished results, grant drafts, IRB materials, and internal code — never paste or upload these into a personal ChatGPT account, a non-Stanford API key, or any tool outside the approved list.
- **Exceptions require prior disclosure and approval.** If a project genuinely requires a tool outside the approved list, you must: Describe the tool and intended use to Jade in writing before use. Confirm what data, if any, would be shared with it. Get explicit sign-off before proceeding.
- **Agentic tools (Claude Code, Codex, Cowork, Gemini CLI, etc.) require extra caution.** Because these tools can read, write, and execute code or take actions autonomously, always: Review generated code/output before running or committing it. Never grant an agentic tool access to production data, credentials, or shared drives without lab PI approval. Disclose if an agentic tool was used to produce analysis code, and document this in commit messages or code comments.
- **When in doubt, ask first.** It's always acceptable — and preferred — to check with Jade before using a new tool, rather than after.
- **You must complete a short quiz and agree to the lab's AI use policy** before using any AI tool for research in the Benjamin-Chung Lab AI Use in Research Permission Form
- **Use of AI tools in research must be documented** in the Benjamin-Chung Lab AI Use in Research Documentation Form

6.1 Three key issues

1. **Learning and skill development.** Heavy reliance on AI coding tools may impede the development of coding fluency and statistical/computational intuition — particularly for trainees who are still

building these skills.
2. **Research accuracy.** Using these tools on datasets, analysis types, or workflows you aren't already familiar with raises the risk of subtle errors going undetected, because you may lack the intuition to recognize when something looks wrong.
3. **Data protection.** These tools can access files on your machine, which introduces new and sometimes subtle data-handling risks in a research context governed by IRB protocols, data use agreements, and participant privacy commitments.

Each is discussed in more detail below, along with suggested safeguards.

6.2 Bottom line

AI coding assistants are a powerful addition to our computational toolkit, but they are not a substitute for the judgment that comes from hands-on coding experience, nor for careful attention to data governance. Use them deliberately: lean on them more for familiar, low-stakes tasks, and less for unfamiliar or high-stakes ones, with unit tests, replication checks, and careful permission review as standing practice regardless of experience level.

When in doubt about whether a workflow is compliant, ask Jade before proceeding.

6.2.1 Research accuracy and the value of experience

In 2026, most seasoned quantitative researchers have been writing code for a decade or more — tens of thousands of cumulative hours. There's a popular "rule" that it takes 10,000 hours to master a new skill– equivalent to 5.5 years of full time work. That experience builds a kind of gut instinct: when you look at code, or watch it run, you develop a sense for what *might* be going wrong, even when nothing throws an obvious error. Examples of silent failures:

- A merge that drops rows unexpectedly
- A factor variable that got silently releveled
- A date field parsed in the wrong timezone

After dealing with these silent errors hundreds or thousands of times, the human brain recognizes the *shape* of a subtle bug before you can necessarily articulate exactly what it is. That experience is what lets seasoned researchers know what safeguards to put in place:

- Which unit tests are worth writing
- Which portions of a work flow require human or independent agentic replication checks
- Which outputs deserve a manual sanity check versus which can be trusted.

AI coding tools do not have this judgment about *your specific project*. They can write tests if asked, but deciding *which* tests matter for a given dataset or analysis is still a human judgment call, grounded in domain and data familiarity.

This means the risk of using these tools is not constant. As your mentor, I am working on developing systems to help you learn when you might be in the **danger zone**. When you're in the danger zone, you are outside your area of established expertise, working with a new data source, an unfamiliar analysis method, or a computational workflow you haven't run by hand before. The challenge is, when you're new to a project or to a field of study, you're always in the **danger zone.**

6.2.2 Impeding learning: an open question

Little to nothing is currently known about the pedagogy of learning to code with AI assistance versus learning to code without it. If you never manually write code — if you never sit with a bug for twenty minutes and slowly figure out that the join key had a leading whitespace — how do you develop the gut instinct described above? We don't know.

It's also possible that this concern will turn out to be less important than it seems. The future of research computing may not involve much hands-on human coding at all. But even if that's true, we don't yet know *how* to teach someone — a rotation student, a new Research Assistant, a first-year PhD student — to "vibe code" well: to direct an AI tool effectively, to catch its mistakes, and to build sound judgment about when to trust its output. **Until the pedagogy of computational research with agentic AI exists, in our lab, we consider risks of agentic AI use to be high for early-career skill-building to be high.**

6.2.3 Data safety in a research context

Claude Code and similar tools run locally and, with your permission, can read and modify files on your laptop — including, potentially, files containing participant-level data governed by IRB protocols and data use agreements.

The practical risk is subtler than "the AI leaks your data." During a typical coding session, the tool may generate tens or even hundreds of permission requests (to read a file, run a command, install a package, access the network, etc.). These prompts are sometimes phrased in dense technical language that is genuinely hard to parse quickly. It is easy to click "allow" reflexively — especially late in a long session — and inadvertently grant access to a raw data directory, or permission to send data over the network (e.g., installing a package, calling an external API), without fully registering what you approved.

This is not primarily a malicious-AI risk; it's a human-factors risk. The tool is doing what you told it to do, but "what you told it to do" is easy to misjudge when permission requests are frequent, fast, and technically dense.

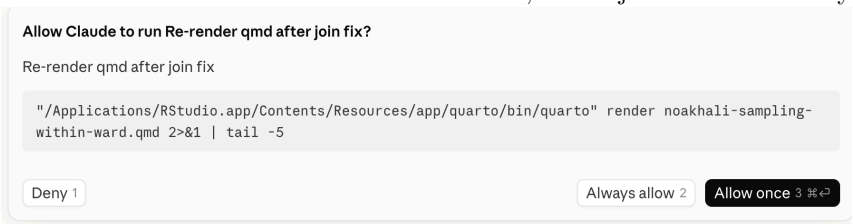
As of June 2026, Stanford's Approved Connectors for AI Services have different different risk categories by different products. Note that Google and OpenAI can be used with High Risk Protected Health Information (PHI) data, while using PHI with Claude requires routing through the Stanford AI API Gateway. If you are not sure whether you your project is using PHI, check with Jade before proceeding.

6.2.4 What these prompts actually look like

Note: the following is based on Claude Code, but other agentic coding tools have similar permission prompts.

The prompt and approval interface differs depending on whether you're working in a terminal or a GUI/editor-based tool, but the underlying pattern is the same:

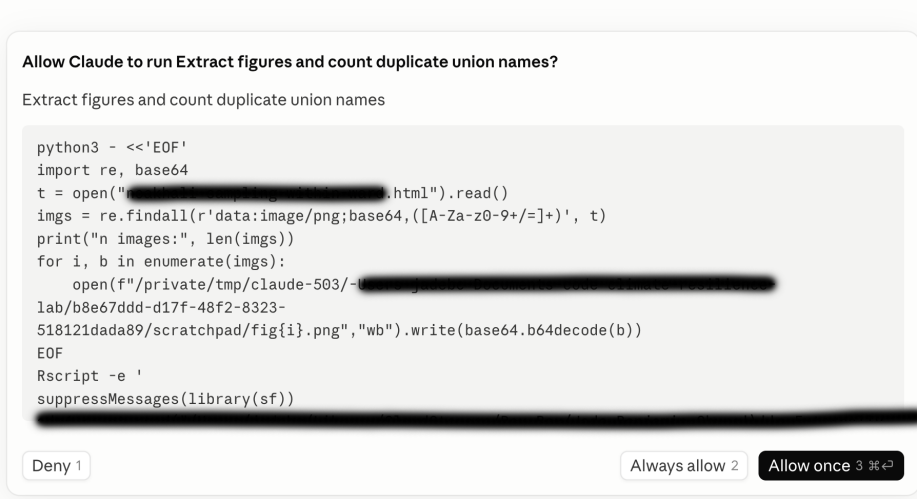
- **AI's own plain-English summary** of the action, not a guarantee of what the command actually does. It's usually accurate, but it can omit exactly the detail — a hardcoded file path, a write to an unexpected directory — that matters for review. Read the actual command or diff underneath, not just the summary.



- **Long commands or file paths that are often truncated.** Claude Code may write a snippet of code and ask you for permission to run it, but it might not be in your project's language, and often these code snippets are not saved in your repository at all. For example, if you're working in an R codebase and ask it to double-check a calculation or validate an assumption, it might quietly reach for Python — writing a small throwaway script to cross-check the logic, run it, read the output, and then report back to you in English. That scratch script often never gets saved as a file you'd see in a pull request; it's more like a post-it note the AI wrote to itself, used once, and threw away. The problem is that if you don't read Python, you have no way to actually verify what that post-it note said or did — you're not reviewing the check, you're reviewing Claude's summary of the check. That might be perfectly fine for something low-stakes, but for a step that's supposed to be your safeguard against a subtle analytic error, it's worth pausing on: if you can't read the language a verification was written in, you aren't really approving the verification — you're approving a description of it, written by the same tool whose work it's supposed to be checking.

Here's an example from a code base entirely written in R, where Claude Code

asked for permission to check for duplicates using a "post-it" note in Python.



- **"Always allow" sits right next to the one-time approval**, in the terminal as option 2 of 3, and in the GUI as a persistent button with its own keyboard shortcut. It's one keystroke away from a single-use approval, which is exactly what makes reflexive over-granting easy, especially late in a long session.

When Claude Code (or a similar AI coding assistant) asks for permission to run a command, recognizing the command itself — not just reading the plain-English gloss above it — is the fastest way to know what you're actually approving. The tables below are organized by risk level.

6.3 Suggested safeguards

6.3.1 For learning and skill development

- **Spend meaningful time writing core analysis code by hand** before leaning on agentic tools for that same type of task — e.g., write your first TMLE or IPW pipeline yourself before asking an AI tool to generate one.
- When using an AI tool for a task you're still learning, **ask it to explain its reasoning**, not just produce output. Treat it as a tutor for code you then re-derive yourself, not a black box.
- Before fully letting agentic AI write a script for you, try writing out detailed pseudo-code – aka, **a step-by-step plan for what the code should do**. Then ask the AI to implement that plan, and check that it actually does what you intended.

Command	What it does	Why it matters
Low-risk commands		
‘ls‘, ‘cat‘, ‘head‘, ‘tail‘	List directory contents; print file contents	
‘cd‘, ‘pwd‘	Change / print working directory	
‘cp‘	Copy files	
‘|‘ (pipe)	Send one command’s output into another as input	
‘&&‘	Run the next command only if the previous one succeeded	
High-risk commands		
‘rm‘	Delete files	No undo. Watch for this near raw data paths
‘rm -rf‘	Force-delete a directory and everything in it, no confirmation	Riskiest pattern to recognize — recursive + force, can wipe a folder tree silently
‘mv‘	Move files	Can silently overwrite a destination file
‘>‘, ‘»‘	Redirect output to a file	‘>‘ silently overwrites the whole target file; easy to miss mid-command
‘sed -i‘, ‘awk‘	Edit files in place / process text	‘-i‘ modifies the original directly — no diff preview
‘chmod‘	Change file permissions	Can make a data file world-readable/writable
‘chown‘	Change file ownership	Rare in normal analysis work; worth a second look
‘find ... -exec‘	Search files, run a command on each match	Can apply ‘rm‘/‘chmod‘ across many files — read the ‘-exec‘ clause carefully
‘curl‘, ‘wget‘	Download content from a URL	Network access — relevant if data shouldn’t leave the machine
‘scp‘, ‘rsync‘	Copy files to/from a remote machine	Data transfer off-machine — scrutinize unfamiliar destinations
‘git push‘, ‘git commit‘	Send code / record a commit	Notice if Claude is committing on your behalf
‘sudo‘	Run with admin privileges	Should almost never be needed for data analysis
‘export‘	Set an environment	May set API

6.3.2 For research accuracy

- **Match AI use to familiarity.** Use these tools most freely for boilerplate, refactoring, and workflows you already know well. Increase manual scrutiny — or avoid heavy AI use — for new datasets, unfamiliar methods, or unfamiliar computational environments.
- **Always require unit tests and replication checks** for AI-assisted analysis code, especially for anything going into a manuscript or grant deliverable. Ask the AI to write these tests, but have a human (ideally someone senior on the project) confirm the tests actually check the right things. Learn more in the chapter Data validation and unit testing in Chapter 12.
- **Spot-check intermediate outputs**, not just final results — row counts after joins/merges, variable distributions before and after recoding, and sample sizes at each analytic step.
- **Independent replication** for key analyses: have a second team member reproduce a critical result from raw data, ideally without heavy reliance on the same AI-generated code path. In the kind of research we do, the code most likely to lead to a lack of replication arises during data processing, i.e., converting survey data to the form needed for analyses. Two reasonable people or agents could make different assumptions, none of which are wrong, that result in different data. Replication brings these differences to light and allows them to be transparently reconciled. Learn more here.

6.3.3 For data protection

- **Never point an AI coding tool at a directory containing raw participant-level data** unless you have specifically confirmed this is permitted under the relevant IRB protocol and data use agreement — and ideally, work with de-identified or synthetic data during code development, applying finished code to real data only in a controlled step.
- **Read permission prompts before approving them**, especially requests to access the network, install packages, or read/write outside the current project folder. When in doubt, ask Jade.
- **Use project-level permission scoping** (e.g., allow-listing specific directories or commands) rather than granting broad or session-wide access.

When in doubt about whether a workflow is compliant, ask Jade before proceeding.

6.4 Know your target journal's AI policy before you start

Before you start a project that will be published in a peer-reviewed journal, keep in mind that you will likely need to disclose any AI assistance in your

methods section, acknowledgments, or cover letter. Journal policies on AI use vary widely (see table below).

- Before relying on AI tools for a substantial part of your workflow, check the current policy for your intended journal to learn what it does and doesn't cover — a policy permitting AI for language editing is not the same as one permitting it for analysis or figure generation.
- Document what tools you use throughout the project so you can disclose them at the time of journal submission.
- Given how quickly tools are changing, you'll need to check the policy again closer to submission.

Here is an example of the type of acknowledgement language you might include in a manuscript:

> Claude (Sonnet 4.6, Anthropic) was used for language editing and improving clarity of expression. Claude Code (Sonnet 4.6, Anthropic) was used to assist with data processing, statistical coding, and figure and table generation. All code was reviewed by two independent team members to verify accuracy and reproducibility. These tools were not used to develop research questions, interpret findings, or draw conclusions. The authors take full responsibility for the interpretations, conclusions, accuracy, and integrity of the work.

The following was generated by Claude in July 2026:

Sources: publisher policy pages accessed July 2026 (springernature.com, journals.plos.org, elsevier.com, authors-bmj-com, academic.oup.com, wiley.com, researcher-resources.acs.org, journals.asm.org, science.org, f1000research.com). Cells marked "not located" should be re-checked directly before submission — this space is evolving fast.

Table 6.1: Journal AI policies as of proposal writing — policies change, verify current policy before heavy AI use.

Publisher	Journals	Policy (not permitted / allowed w/ disclosure / not addressed)
Policy documented		
Springer Nature	Nature, Nat. Comms/Med/Health, Sci. Reports, BMC, Discover Env.	Authorship; copy-edit w/ disclosure; name tool + use in Methods
PLOS	Glob. Public Health, Medicine, NTD	Authorship; tool/use/validation disclosed in Methods/Ack.
Elsevier/Lancet	Lancet GH/PH, eBioMed, eClinicalMed, Vaccine, IJHEH, Heliyon	Authorship; copy-edit w/ disclosure; declaration before references
BMJ	BMJ Open, BMJ Glob. Health	Authorship; described in manuscript + flagged at submission
Oxford UP	AJE, IJE, Epi Reviews, CID, JID, OFID	Authorship; coding/analysis w/ disclosure; cover letter + Methods/Ack.
Wiley	Stats in Med, TMIH, Paediatric & Perinatal Epi	Authorship; copy-edit w/ disclosure; data alteration; Methods/Ack. if substantial
ACS	Env. Sci. & Tech.	Authorship; disclosed in Ack., Methods if substantial
ASM	Applied & Env. Micro.	Authorship; cover letter + Materials & Methods
AAAS/Science	Science	Authorship; cover letter + Ack. + full prompts in Methods
F1000	Gates Open Research	Authorship; not detailed — check w/ editor
Policy not yet located — check before use		
ASTMH	AJTMH	Not located — verify before use
APHA	AJPH	Not located; AI images must be disclosed
eLife	eLife	Not located — verify before use
NIEHS	EHP	Not located — verify before use
Wolters Kluwer	Epidemiology	Not located — verify before use

Chapter 7

Code repositories

By Kunal Mishra, Jade Benjamin-Chung, and Stephanie Djajadi

Each study has at least one code repository that typically holds R code, shell scripts with Unix code, and research outputs (results .RDS files, tables, figures). Repositories may also include datasets. This chapter outlines how to organize these files. Adhering to a standard format makes it easier for us to efficiently collaborate across projects.

7.1 Project Structure

We recommend the following directory structure:

```
0-run-project.sh
0-config.R
0-functions/
    0-base-functions.R
    0-plot-functions.R
    ...
1 - Data-Management/
    0-prep-data.sh
    1-prep-cdph-fluseas.R
    2-prep-absentee.R
    ...
2 - Analysis/
    0-run-analysis.sh
    1 - Absentee-Mean/
        1-absentee-mean-primary.R
        2-absentee-mean-negative-control.R
        ...
    2 - Absentee-Positivity-Check/
```

```
    3 - Absentee-P1/
    4 - Absentee-P2/
3 - Figures/
    0-run-figures.sh
    ...
4 - Tables/
    0-run-tables.sh
    ...
5 - Results/
    1 - Absentee-Mean/
        1-absentee-mean-primary.RDS
        2-absentee-mean-negative-control.RDS
    ...
.gitignore
.Rproj
```

For brevity, not every directory is "expanded", but we can glean some important takeaways from what we *do* see.

7.2 `0-functions` folder

The `0-functions/` folder contains reusable helper functions that are shared across multiple scripts in the project. These are sourced in `0-config.R` so they are available to every script in the pipeline. `0-base-functions.R` typically contains functions for data cleaning, regression wrappers (e.g., fitting a model with robust standard errors and formatting the output), and common data manipulation tasks. `0-plot-functions.R` contains functions for generating figures with consistent formatting, such as custom ggplot themes, color palettes, and wrappers for common plot types used across the project.

7.3 `.Rproj` files

An "R Project" can be created within RStudio by going to `File >> New Project`. Depending on where you are with your research, choose the most appropriate option. This will save preferences, working directories, and even the results of running code/data (though I'd recommend starting from scratch each time you open your project, in general). Then, ensure that whenever you are working on that specific research project, you open your created project to enable the full utility of `.Rproj` files. This also automatically sets the directory to the top level of the project.

7.4 Configuration ('config') File

This is the single most important file for your project to ensure reproducibility. It loads all libraries, sources shared functions from `0-functions/`, declares global variables, and defines file paths for data, figures, and tables. *Every other file in the project* will begin with `source("0-config.R")`, which means changes made here automatically propagate throughout the project. For example, if you need to rename a data directory or switch from a downsampled dataset to the full data, you only modify the path in `0-config.R` rather than in every script that references it.

The config file also handles differences between collaborators' machines. Since Box syncs to a different local path on each person's computer, the config detects which user is running the code and sets the Box root path accordingly. Local paths within the project repository (for data, figures, and tables) use the `here` package, which assumes that users have opened the `.Rproj` file, setting the working directory to the top level of the project.

Here is an example config file:

```
#-------------------------------------
# Name of study

# configure data directories
# source base functions
# load libraries
#-------------------------------------
library(tidyverse)
library(here)
library(ggplot2)
library(gridExtra)
library(reshape2)
library(assertthat)
library(testthat)

if(Sys.getenv("LOGNAME")=="person1"){
  box_root_path = "<FILL IN PERSON 1'S COMPUTER'S BOX PATH>"
}

if(Sys.getenv("LOGNAME")=="person2"){
  box_root_path = "<FILL IN PERSON 2'S COMPUTER'S BOX PATH>"
}

# Box paths
box_path <- paste0(box_root_path, "Housing-DHS-analysis/")
box_path_raw_data <- paste0(box_path, "raw-data/")
box_path_processed_data <- paste0(box_path, "processed-data/")
```

```
# Local directories
data_path = here::here("data/")
figure_path = here::here("figures/")
table_path = here::here("tables/")
```

7.5 Order Files and Directories

At the end of the project, we recommend ordering the files in order of how they must be run and then numbering them within sub-folders. This makes the jumble of alphabetized file names much more coherent and places similar code and files next to one another. This also helps us understand how data flows from start to finish and allows us to easily map a script to its output (i.e. `2 - Analysis/1 - Absentee-Mean/1-absentee-mean-primary.R` => `5 - Results/1 - Absentee-Mean/1-absentee-mean-primary.RDS`). If you take nothing else away from this guide, this is the single most helpful suggestion to make your workflow more coherent. Often the particular order of files will be in flux until an analysis is close to completion. At that time it is important to review file order and naming and reproduce everything prior to drafting a manuscript.

7.6 Using Bash scripts to ensure reproducibility

Bash scripts are useful components of a reproducible workflow. At many of the directory levels (i.e. in `3 - Analysis`), there is a bash script that runs each of the analysis scripts. This is exceptionally useful when data "upstream" changes – you simply run the bash script. See the Unix Chapter for further details.

After running bash scripts, `.Rout` log files will be generated for each script that has been executed. It is important to check these files. Scripts may appear to have run correctly in the terminal, but checking the log files is the only way to ensure that everything has run completely.

Chapter 8

Coding practices

by Kunal Mishra, Jade Benjamin-Chung, Stephanie Djajadi, and Iris Tong

8.1 Organizing scripts

Just as your data "flows" through your project, data should flow naturally through a script. Very generally, you want to:

1. describe the work completed in the script in a comment header
2. source your configuration file (`0-config.R`)
3. load all your data
4. do all your analysis/computation
5. save your data.

Each of these sections should be "chunked together" using comments. See this file for a good example of how to cleanly organize a file in a way that follows this "flow" and functionally separate pieces of code that are doing different things.

8.2 Documenting your code

8.2.1 File headers

Every file in a project should have a header that allows it to be interpreted on its own. It should include the name of the project and a short description for what this file (among the many in your project) does specifically. You may optionally wish to include the inputs and outputs of the script as well, though the next section makes this significantly less necessary.

```
################################################################################
# @Organization - Example Organization
# @Project - Example Project
```

```
# @Description - This file is responsible for [...]
################################################################################
```

8.2.2 Sections and subsections

Rstudio (v1.4 or more recent) supports the use of Sections and Subsections. You can easily navigate through longer scripts using the navigation pane in RStudio, as shown on the right below.

```
# Section -------

## Subsection -------

### Sub-subsection -------
```

8.2.3 Code folding

Consider using RStudio's code folding feature to collapse and expand different sections of your code. Any comment line with at least four trailing dashes (-), equal signs (=), or pound signs (#) automatically creates a code section. For example:

8.2.4 Comments in the body of your script

Commenting your code is an important part of reproducibility and helps document your code for the future. When things change or break, you'll be thankful for comments. There's no need to comment excessively or unnecessarily, but a comment describing what a large or complex chunk of code does is always helpful. See this file for an example of how to comment your code and notice that comments are always in the form of:

```
# This is a comment -- first letter is capitalized and spaced
away from the pound sign
```

8.2.5 Function documentation

Every function you write must include a header to document its purpose, inputs, and outputs. For any reproducible workflows they are essential because R is dynamically typed. This means you can pass a `string` into an argument that is meant to be a `data.table`, or a `list` into an argument meant for a `tibble`. It is the responsibility of a function's author to document what each argument is meant to do and its basic type.

We use roxygen2-style documentation headers (`#'` comments with `@param`, `@return` tags). This is the standard convention used in R package development, which means students will encounter it throughout the R ecosystem. It is also structured enough to be parsed programmatically, which supports automated code review. Here is an example:

```
##############################################
##############################################
#' Calculate flu season means by site
#'
#' Make a dataframe with rows for flu season and site
#' and the number of patients with an outcome, the total patients,
#' and the percent of patients with the outcome.
#'
#' @param data A data frame with variables flu_season, site, studyID, and yname.
#' @param yname A string for the outcome name.
#' @param silent A boolean specifying whether the function should suppress
#'   console output. Default: TRUE.
#'
#' @return A data frame with columns for flu season, site, outcome count,
#'   total count, and percent.
calc_fluseas_mean <- function(data, yname, silent = TRUE) {
function code here
}
```

The header tells you what the function does, its various inputs, and how you might go about using the function to do what you want. Also notice that all optional arguments (i.e. ones with pre-specified defaults) follow arguments that require user input.

- **Note**: As someone trying to call a function, it is possible to access a function's documentation (and internal code) by `CMD-Left-Click`ing the function's name in RStudio
- **Note**: Depending on how important your function is, the complexity of your function code, and the complexity of different types of data in your project, you can also add "type-checking" to your function with the `assertthat::assert_that()` function. You can, for example, `assert_that(is.data.frame(statistical_input))`, which will ensure that collaborators or reviewers of your project attempting to use your function are using it in the way that it is intended by calling it with (at the minimum) the correct type of arguments. You can extend this to ensure that certain assumptions regarding the inputs are fulfilled as well (i.e. that `time_column`, `location_column`, `value_column`, and `population_column` all exist within the `statistical_input` tibble).

8.3 Object naming

Generally we recommend using nouns for objects and verbs for functions. This is because functions are performing actions, while objects are not.

Try to make your variable names both more expressive and more explicit. Being a bit more verbose is useful and easy in the age of autocomple-

tion! For example, instead of naming a variable `vaxcov_1718`, try naming it `vaccination_coverage_2017_18`. Similarly, `flu_res` could be named `absentee_flu_residuals`, making your code more readable and explicit.

- For more help, check out Be Expressive: How to Give Your Variables Better Names

We recommend you use **Snake_Case**.

- Base R allows `.` in variable names and functions (such as `read.csv()`), but this goes against best practices for variable naming in many other coding languages. For consistency's sake, `snake_case` has been adopted across languages, and modern packages and functions typically use it (i.e. `readr::read_csv()`). As a very general rule of thumb, if a package you're using doesn't use `snake_case`, there may be an updated version or more modern package that *does*, bringing with it the variety of performance improvements and bug fixes inherent in more mature and modern software.

- **Note**: you may also see `camelCase` throughout the R code you come across. This is *okay* but not ideal – try to stay consistent across all your code with `snake_case`.

- **Note**: again, its also worth noting there's nothing inherently wrong with using `.` in variable names, just that it goes against style best practices that are cropping up in data science, so its worth getting rid of these bad habits now.

8.4 Function calls

In a function call, use "named arguments" and put each argument on a separate line to make your code more readable.

Here's an example of what not to do when calling the function a function `calc_fluseas_mean` (defined above):

```
mean_Y = calc_fluseas_mean(flu_data, "maari_yn", FALSE)
```

And here it is again using the best practices we've outlined:

```
mean_Y = calc_fluseas_mean(
  data = flu_data,
  yname = "maari_yn",
  silent = FALSE
)
```

8.5 The here package

The `here` package is one great R package that helps multiple collaborators deal with the mess that is working directories within an R project structure. Let's say we have an R project at the path `/home/oski/Some-R-Project`. My collaborator might clone the repository and work with it at some other path, such as `/home/bear/R-Code/Some-R-Project`. Dealing with working directories and paths explicitly can be a very large pain, and as you might imagine, setting up a Config with paths requires those paths to flexibly work for all contributors to a project. This is where the `here` package comes in and this a great vignette describing it.

8.6 Reading/Saving Data

8.6.1 `.RDS` vs `.RData` Files

One of the most common ways to load and save data in Base R is with the `load()` and `save()` functions to serialize multiple objects in a single `.RData` file. The biggest problems with this practice include an inability to control the names of things getting loaded in, the inherent confusion this creates in understanding older code, and the inability to load individual elements of a saved file. For this, we recommend using the RDS format to save R objects.

- **Note**: if you have many related R objects you would have otherwise saved all together using the `save` function, the functional equivalent with `RDS` would be to create a (named) list containing each of these objects, and saving it.

8.6.2 CSVs

Once again, the `readr` package as part of the Tidvyerse is great, with a much faster `read_csv()` than Base R's `read.csv()`. For massive CSVs (> 5 GB), you'll find `data.table::fread()` to be the fastest CSV reader in any data science language out there. For writing CSVs, `readr::write_csv()` and `data.table::fwrite()` outclass Base R's `write.csv()` by a significant margin as well.

8.7 Tidyverse

R has two broad dialects you'll encounter: base R, which is the syntax built into the language, and the tidyverse, a collection of packages (including `dplyr`, `tidyr`, `ggplot2`, and `readr`) that provide an alternative, more consistent syntax for data manipulation and visualization. If you learned R in a short course, you may have used a mix of both without realizing it. Base R uses syntax like `df[df$age < 5, ]` and `merge()`, while tidyverse uses pipes and verbs like `df %>% filter(age < 5)` and `left_join()`.

Both work, but our lab standardizes on tidyverse because the syntax is more readable and consistent across operations, which makes code easier to review and reproduce. The table below maps common base R patterns to their tidyverse equivalents for reference. For large data pipelines, `data.table` is a performant alternative. If you encounter base R patterns in legacy code or external packages, the reference table below maps common translations. Hadley Wickham's R for Data Science is a good resource for learning the Tidyverse ecosystem.

The following list is not exhaustive, but is a compact reference for common base R to Tidyverse translations:

Base R	Better Style, Performance, and Utility
—	—
`read.csv()`	`readr::read_csv()` or `data.table::fread()`
`write.csv()`	`readr::write_csv()` or `data.table::fwrite()`
`readRDS`	`readr::read_rds()`
`saveRDS()`	`readr::write_rds()`
—	—
`data.frame()`	`tibble::tibble()` or `data.table::data.table()`
`rbind()`	`dplyr::bind_rows()`
`cbind()`	`dplyr::bind_cols()`
`df$some_column`	`df %>% dplyr::pull(some_column)`
`df$some_column = ...`	`df %>% dplyr::mutate(some_column = ...)`
`df[get_rows_condition,]`	`df %>% dplyr::filter(get_rows_condition)`
`df[,c(col1, col2)]`	`df %>% dplyr::select(col1, col2)`
`merge(df1, df2, by = ..., all.x = ..., all.y = ...)`	`df1 %>% dplyr::left_join(df2, by = ...)` or `dplyr::full_join` or `dplyr::inner_join` or `dplyr::right_join`
—	—
`str()`	`dplyr::glimpse()`
`grep(pattern, x)`	`stringr::str_which(string, pattern)`
`gsub(pattern, replacement, x)`	`stringr::str_replace(string, pattern, replacement)`
`ifelse(test_expression, yes, no)`	`if_else(condition, true, false)`

Base R	Better Style, Performance, and Utility
Nested: `ifelse(test_expression1, yes1, ifelse(test_expression2, yes2, ifelse(test_expression3, yes3, no)))`	`case_when(test_expression1 ~ yes1, test_expression2 ~ yes2, test_expression3 ~ yes3, TRUE ~ no)`
`proc.time()`	`tictoc::tic()` and `tictoc::toc()`
`stopifnot()`	`assertthat::assert_that()` or `assertthat::see_if()` or `assertthat::validate_that()`

For a more extensive set of syntactical translations to Tidyverse, you can check out this document.

Working with Tidyverse within functions can be somewhat of a pain due to non-standard evaluation (NSE) semantics. If you're an avid function writer, we'd recommend checking out the following resources:

- Tidy Eval in 5 Minutes (video)
- Tidy Evaluation (e-book)
- Data Frame Columns as Arguments to Dplyr Functions (blog)
- Standard Evaluation for *_join (stackoverflow)
- Programming with dplyr (package vignette)

8.8 Coding with R and Python

Some projects may require both R and Python — for example, using Python for machine learning (e.g., `scikit-learn`, `pytorch`) and R for statistical analysis and visualization. There are two main strategies for interoperability:

Exchanging data via Apache Arrow / Parquet. The Arrow and Parquet formats allow fast, zero-copy data exchange between R and Python with no loss of type information. In R, use the `arrow` package to read and write Parquet files:

```
# In R
library(arrow)
write_parquet(my_data, here::here("data", "my_data.parquet"))
my_data <- read_parquet(here::here("data", "my_data.parquet"))
```

```
# In Python
import pandas as pd
df = pd.read_parquet("data/my_data.parquet")
df.to_parquet("data/my_data.parquet")
```

Parquet is preferred over CSV for R-Python workflows because it preserves column types, handles dates correctly, and is much faster for large datasets.

Calling Python from R with `reticulate`. The `reticulate` package lets you call Python functions directly from R, which can be useful when you want to use a Python library without leaving your R workflow. However, this adds complexity to reproducible environments (you need to manage both `renv` and a Python virtual environment), so use it judiciously.

Note: The `feather` package, which was previously recommended for R-Python data exchange, has been superseded by Arrow/Parquet and should no longer be used.

8.9 Repeating analyses with different variations

In many cases, we will need to apply our modeling on different combinations of interests (outcomes, exposures, etc.). We can certainly use a `for` loop to repeat the execution of a wrapper function, but generally, `for` loops request high memory usage and produce the results in long computation time.

Fortunately, R has some functions which implement looping in a compact form to help repeating your analyses with different variations (subgroups, outcomes, covariate sets, etc.) with better performances.

8.9.1 `lapply()` and `sapply()`

`lapply()` is a function in the base R package that applies a function to each element of a list and returns a list. It's typically faster than `for`. Here is a simple generic example:

```
result <- lapply(X = mylist, FUN = func)
```

There is another very similar function called `sapply()`. It also takes a list as its input, but if the output of the `func` is of the same length for each element in the input list, then `sapply()` will simplify the output to the simplest data structure possible, which will usually be a vector.

8.9.2 `mapply()` and `pmap()`

Sometimes, we'd like to employ a wrapper function that takes arguments from multiple different lists/vectors. Then, we can consider using `mapply()` from the base R package or `pmap()` from the `purrr` package.

Please see the simple specific example below where the two input lists are of the same length and we are doing a pairwise calculation:

```
mylist1 = list(0:3)
mylist2 = list(6:9)
mylists = list(mylist1, mylist2)

square_sum <- function(x, y) {
```

```
  x^2 + y^2
}

#Use `mapply()`
result1 <- mapply(FUN = square_sum, mylist1, mylist2)

#Use `pmap()`
library(purrr)
result2 <- pmap(.l = mylists, .f = square_sum)

#unlist(as.list(result1)) = result2 = [36 50 68 90]
```

There are two major differences between `mapply()` and `pmap()`. The first difference is that `mapply()` takes seperate lists as its input arguments, while `pmap()` takes a list of list. Secondly, the output of `mapply()` will be in the form of a matrix or an array, but `pmap()` produces a list directly.

However, when **the input lists are of different lengths AND/OR the wrapper function doesn't take arguments in pairs, `mapply()` and `pmap()` may not give the preferable results.**

Both `mapply()` and `pmap()` will recycle shorter input lists to match the length of the longest input list. Assume that now `mylist2 = list(6:12)`. Then, `pmap(mylists, square_sum)` will generate `[36 50 68 90 100 122 148]` where elements 0, 1, and 2 are recycled to match 10, 11, and 12. And it will return an error message that "longer object length is not a multiple of shorter object length."

Thus, unless the recycling pattern described above is desirable feature for a certain experiment design, **when the input lists are of different lengths, the best practice is probably to use `lapply()` and then combine the results.**

Here is an example where we'd like to find the `square_sum` for every element combination of `mylist1` and `mylist2`.

```
mylist1 <- list(0:3)
mylist2 <- list(6:12)

square_sum <- function(x, y) {
  x^2 + y^2
}

results <- list()

for (i in seq_along(mylist1[[1]])) {
  result <- lapply(X = mylist2, FUN = function(y) square_sum(mylist1[[1]][i], y))
  results[[i]] <- result
}
```

This example doesn't work in the way that 0 is paired to 6, 1 is paired to 7, and so on. Instead, every element in `mylist1` will be paired with every element in `mylist2`. Thus, the "unlisted" results from the example will have $4 * 7 = 28$ elements.

We can use `flatten()` or `unlist()` functions to decrease the depths of our results. If the results are data frames, then we will need to use `bind_rows()` to combine them.

8.9.3 Parallel processing with `parallel` and `future` packages

One big drawback of `lapply()` is its long computation time, especially when the list length is long. Fortunately, computers nowadays must have multiple cores which makes parallel processing possible to help make computation much faster.

Assume you have a list called `mylist` of length 1000, and `lapply(X = mylist, FUN = func)` applies the function to each of the 1000 elements one by one in T seconds. If we could execute the `func` in n processors simultaneously, then ideally, we would shrink the computation time to T/n seconds.

In practice, using functions under the `parallel` and the `future` packages, we can split `mylist` into smaller chunks and apply the function to each element of the several chunks in parallel in different cores to significantly reduce the run time.

8.9.3.1 `parLapply()`

Below is a generic example of `parLapply()`:

```
library(parallel)

# Set how many processors will be used to process the list and make cluster
n_cores <- 4
cl <- makeCluster(n_cores)

#Use parLapply() to apply func to each element in mylist
result <- parLapply(cl = cl, x = mylist, FUN = func)

#Stop the parallel processing
stopCluster(cl)
```

Let's still assume `mylist` is of length 1000. The `parLapply` above splits `mylist` into 4 sub-lists each of length 250 and applies the function to the elements of each sub-list in parallel. To be more specific, first apply the function to element 1, 251, 501, 751; second apply to element 2, 252, 502, 752; so on and so forth. As such, the computation time will be greatly reduced.

You can use `parallel::detectCores()` to test how many cores your machine has and to help decide what to put for `n_cores`. It would be a good idea to leave at least one core free for the operating system to use.

We will always start `parLapply()` with `makeCluster()`. **`stopCluster()` is not fully necessary but follows the best practices.** If not stopped, the processing will continue in the back end and consuming the computation capacity for other software in your machine. But keep in mind that stopping the cluster is similar quitting R, meaning that you will need to re-load the packages needed when you need to do parallel processing use `parLapply()` again.

8.9.3.2 `future.lapply()`

Below is a generic example of `future.lapply()`:

```
library(future)
library(future.apply)

# First, plan how the future_lapply() will be resolved
future::plan(
  multisession, workers = future::availableCores() - 1
)

# Use future_lapply() to apply func to each element in mylist
future_lapply(x = mylist, FUN = func)
```

Here, `future::availableCores()` checks how many cores your machine has. Similar to `parLapply()` showed above, `future_lapply()` parallelizes the computation of `lapply()` by executing the function `func` simultaneously on different sub-lists of `mylist`.

8.10 Reviewing Code

Code review is one of the most important quality control steps in our workflow. Every analysis that will be used in a manuscript, report, or presentation should be reviewed by at least one other person before results are considered final. GitHub pull requests are our primary mechanism for code review.

8.10.1 Why we review code

Even careful, experienced analysts make mistakes. Common issues caught in review include: incorrect merge keys that silently drop or duplicate observations, filter conditions that exclude the wrong subgroups, mislabeled variables in results tables, and hardcoded values that should be parameterized. In an epidemiology context, these errors can change point estimates, confidence intervals, or conclusions — and they are often invisible in the final output without examining the code itself.

8.10.2 What to look for in a code review

When reviewing a pull request, check for the following:

Correctness

- Does the code do what the analyst intends? Read the description of the task (e.g., the Trello card) and verify the code implements it.
- Check merge/join operations carefully: Are the join keys correct? Does the resulting number of rows make sense? Are there unexpected duplicates or dropped observations?
- Check filter and subset operations: Are the conditions correct? Is the remaining sample size plausible?
- Do computed variables (e.g., rates, differences, person-time) use the correct formula?
- Are results consistent with what you'd expect given the data? Look for red flags like zero standard errors, wildly implausible estimates, or sample sizes that don't match expectations.

Reproducibility

- Does the script follow the standard flow (header → config → load → analyze → save)?
- Are file paths relative (using `here()`) rather than absolute?
- Are there any hardcoded values that should come from the data or config file?
- Does the code run end-to-end without manual intervention?
- Are random seeds set where relevant (e.g., bootstrapping, cross-validation)?

Clarity and style

- Is the code well-commented? Could someone unfamiliar with the project understand what each major section does?
- Are variable and function names descriptive?
- Are functions documented with headers (see Function documentation above)?
- Does the code follow our style conventions (snake_case, named arguments, tidyverse)?

Data integrity

- Are assertions or sanity checks present where they should be (see the [Testing and validation] chapter)?
- Is the code careful about missing data — e.g., does it use `na.rm = TRUE` intentionally vs. ignoring missingness by default?
- Are there any places where PHI or sensitive data could be accidentally exposed (e.g., printing raw IDs, writing intermediate files without de-identification)?

8.10.3 How to give feedback

Be specific and constructive. Instead of "this looks wrong," say "the left join on line 45 uses `household_id` but I think it should use `individual_id` since we want one row per person." When something is a suggestion rather than a required change, label it as such (e.g., "Optional: you could simplify this with `across()`").

GitHub's review interface lets you leave comments on specific lines, which is much more useful than general comments. Use "Request changes" when there are issues that must be fixed before merging, and "Approve" when the code is ready (or when only minor optional suggestions remain).

8.10.4 When to approve

Approve a pull request when you are confident that the code is correct, reproducible, and readable. You do not need to verify every line of arithmetic, but you should understand the overall logic and have checked the high-risk operations (joins, filters, key computations). If you're unsure about something, ask — it's better to have a conversation than to approve code you don't understand.

8.10.5 Creating a Pull Request Template

Follow the instructions below to add a pull request template to a repository. More details can be found at this GitHub link.

1. On GitHub, navigate to the main page of the repository.
2. Above the file list, click `Create new file`.
3. Name the file `pull_request_template.md`. GitHub will not recognize this as the template if it is named anything else. The file must be on the `master` branch.
    1. To store the file in a hidden directory instead of the main directory, name the file `.github/pull_request_template.md`.
4. In the body of the new file, add your pull request template. This could include:
    - A summary of the changes proposed in the pull request
    - How the change has been tested
    - @mentions of the person or team responsible for reviewing proposed changes

Here is an example pull request template.

```
# Description

## Summary of change

Please include a summary of the change, including any new functions added and example usage.
```

```
## Testing

Describe how you tested the code:
- [ ] Script runs end-to-end without errors
- [ ] Output sample sizes match expectations
- [ ] Key estimates are plausible
- [ ] Assertions/sanity checks pass

## Link to Spec

Please include a link to the Trello card or Google document with details of the task.

## Who should review the pull request?
@ ...
```

Chapter 9

Coding style

by Kunal Mishra, Jade Benjamin-Chung, and Stephanie Djajadi

9.1 Comments

1. **File Headers** - Every file in a project should have a header that allows it to be interpreted on its own. It should include the name of the project and a short description for what this file (among the many in your project) does specifically. You may optionally wish to include the inputs and outputs of the script as well, though the next section makes this significantly less necessary.

```
################################################################################
# @Organization - Example Organization
# @Project - Example Project
# @Description - This file is responsible for [...]
################################################################################
```

2. **File Structure** - Just as your data "flows" through your project, data should flow naturally through a script. Very generally, you want to 1) source your config => 2) load all your data => 3) do all your analysis/computation => save your data. Each of these sections should be "chunked together" using comments. See this file for a good example of how to cleanly organize a file in a way that follows this "flow" and functionally separate pieces of code that are doing different things.
    - **Note**: If your computer isn't able to handle this workflow due to RAM or requirements, modifying the ordering of your code to accomodate it won't be ultimately helpful and your code will be fragile, not to mention less readable and messy. You need to look into high-performance computing (HPC) resources in this case.

3. **Single-Line Comments** - Commenting your code is an important part of reproducibility and helps document your code for the future. When things change or break, you'll be thankful for comments. There's no need to comment excessively or unnecessarily, but a comment describing what a large or complex chunk of code does is always helpful. See this file for an example of how to comment your code and notice that comments are always in the form of:

```
# This is a comment -- first letter is capitalized and spaced
away from the pound sign
```

4. **Multi-Line Comments** - Occasionally, multi-line comments are necessary. Don't add line breaks manually to a single-line comment for the purpose of making it "fit" on the screen. Instead, in RStudio > Tools > Global Options > Code > "Soft-wrap R source files" to have lines wrap around. Format your multi-line comments like the file header from above.

9.2 Line breaks

- For `ggplot` calls and `dplyr` pipelines, do not crowd single lines. Here are some nontrivial examples of "beautiful" pipelines, where beauty is defined by coherence:

```
# Example 1
school_names = list(
  OUSD_school_names = absentee_all %>%
    filter(dist.n == 1) %>%
    pull(school) %>%
    unique %>%
    sort,

  WCCSD_school_names = absentee_all %>%
    filter(dist.n == 0) %>%
    pull(school) %>%
    unique %>%
    sort
)

# Example 2
absentee_all = fread(file = raw_data_path) %>%
  mutate(program = case_when(schoolyr %in% pre_program_schoolyrs ~ 0,
                             schoolyr %in% program_schoolyrs ~ 1)) %>%
  mutate(period = case_when(schoolyr %in% pre_program_schoolyrs ~ 0,
                            schoolyr %in% LAIV_schoolyrs ~ 1,
                            schoolyr %in% IIV_schoolyrs ~ 2)) %>%
  filter(schoolyr != "2017-18")
```

And of a complex `ggplot` call:

```
# Example 3
ggplot(data=data,
       mapping=aes_string(x="year", y="rd", group=group)) +

  geom_point(mapping=aes_string(col=group, shape=group),
             position=position_dodge(width=0.2),
             size=2.5) +

  geom_errorbar(mapping=aes_string(ymin="lb", ymax="ub", col=group),
                position=position_dodge(width=0.2),
                width=0.2) +

  geom_point(position=position_dodge(width=0.2),
             size=2.5) +

  geom_errorbar(mapping=aes(ymin=lb, ymax=ub),
                position=position_dodge(width=0.2),
                width=0.1) +

  scale_y_continuous(limits=limits,
                     breaks=breaks,
                     labels=breaks) +

  scale_color_manual(std_legend_title,values=cols,labels=legend_label) +
  scale_shape_manual(std_legend_title,values=shapes, labels=legend_label) +
  geom_hline(yintercept=0, linetype="dashed") +
  xlab("Program year") +
  ylab(yaxis_lab) +
  theme_complete_bw() +
  theme(strip.text.x = element_text(size = 14),
        axis.text.x = element_text(size = 12)) +
  ggtitle(title)
```

Imagine (or perhaps mournfully recall) the mess that can occur when you don't strictly style a complicated `ggplot` call. Trying to fix bugs and ensure your code is working can be a nightmare. Now imagine trying to do it with the same code 6 months after you've written it. Invest the time now and reap the rewards as the code practically explains itself, line by line.

9.3 Automated Tools for Style and Project Workflow

9.3.1 Styling

1. **Code Autoformatting** - RStudio includes a fantastic built-in utility (keyboard shortcut: `CMD-Shift-A`) for autoformatting highlighted chunks of code to fit many of the best practices listed here. It generally makes code more readable and fixes a lot of the small things you may not feel like fixing yourself. Try it out as a "first pass" on some code of yours that *doesn't* follow many of these best practices!

2. **Assignment Aligner** - A cool R package allows you to very powerfully format large chunks of assignment code to be much cleaner and much more readable. Follow the linked instructions and create a keyboard shortcut of your choosing (recommendation: `CMD-Shift-Z`). Here is an example of how assignment aligning can dramatically improve code readability:

```
# Before
OUSD_not_found_aliases = list(
  "Brookfield Village Elementary" = str_subset(string = OUSD_school_shapes$schnam, patt
  "Carl Munck Elementary" = str_subset(string = OUSD_school_shapes$schnam, pattern = "
  "Community United Elementary School" = str_subset(string = OUSD_school_shapes$schnam
  "East Oakland PRIDE Elementary" = str_subset(string = OUSD_school_shapes$schnam, patt
  "EnCompass Academy" = str_subset(string = OUSD_school_shapes$schnam, pattern = "EnCo
  "Global Family School" = str_subset(string = OUSD_school_shapes$schnam, pattern = "G
  "International Community School" = str_subset(string = OUSD_school_shapes$schnam, pa
  "Madison Park Lower Campus" = "Madison Park Academy TK-5",
  "Manzanita Community School" = str_subset(string = OUSD_school_shapes$schnam, patter
  "Martin Luther King Jr Elementary" = str_subset(string = OUSD_school_shapes$schnam, 
  "PLACE @ Prescott" = "Preparatory Literary Academy of Cultural Excellence",
  "RISE Community School" = str_subset(string = OUSD_school_shapes$schnam, pattern = "
)

# After
OUSD_not_found_aliases = list(
  "Brookfield Village Elementary"      = str_subset(string = OUSD_school_shapes$schnam
  "Carl Munck Elementary"              = str_subset(string = OUSD_school_shapes$schnam
  "Community United Elementary School" = str_subset(string = OUSD_school_shapes$schnam
  "East Oakland PRIDE Elementary"      = str_subset(string = OUSD_school_shapes$schnam
  "EnCompass Academy"                  = str_subset(string = OUSD_school_shapes$schnam
  "Global Family School"               = str_subset(string = OUSD_school_shapes$schnam
  "International Community School"     = str_subset(string = OUSD_school_shapes$schnam
  "Madison Park Lower Campus"          = "Madison Park Academy TK-5",
  "Manzanita Community School"         = str_subset(string = OUSD_school_shapes$schnam
  "Martin Luther King Jr Elementary"   = str_subset(string = OUSD_school_shapes$schnam
  "PLACE @ Prescott"                   = "Preparatory Literary Academy of Cultural Exce
```

```
  "RISE Community School"              = str_subset(string = OUSD_school_shapes$schnam, pattern =
)
```

3. **StyleR** - Another cool R package from the Tidyverse that can be powerful and used as a first pass on entire projects that need refactoring. The most useful function of the package is the `style_dir` function, which will style all files within a given directory. See the function's documentation and the vignette linked above for more details.
    - **Note**: The default Tidyverse styler is subtly different from some of the things we've advocated for in this document. Most notably we differ with regards to the assignment operator (`<-` vs `=`) and number of spaces before/after "tokens" (i.e. Assignment Aligner add spaces before `=` signs to align them properly). For this reason, we'd recommend the following: `style_dir(path = ..., scope = "line_breaks", strict = FALSE)`. You can also customize StyleR even more if you're really hardcore.
    - **Note**: As is mentioned in the package vignette linked above, StyleR modifies things *in-place*, meaning it overwrites your existing code and replaces it with the updated, properly styled code. This makes it a good fit on projects *with version control*, but if you don't have backups or a good way to revert back to the intial code, I wouldn't recommend going this route.
4. **Linter** - Linters are programming tools that check adherence to a given style, syntax errors, and possible semantic issues. The R linter, called `lintr`, can be found in this package. It helps keep files consistent across different authors and even different organizations. For example, it notifies you if you have unused variables, global variables with no visible binding, not enough or superflous whitespace, and improper use of parentheses or brackets. A list of its other purposes can be found in this link, and most guidelines are based on Hadley Wickham's R Style Guide.
    - **Note**: You can customize your settings to set defaults or to exclude files. More details can be found here.
    - **Note**: The `lintr` package goes hand in hand with the `styler` package. The styler can be used to automatically fix the problems that the lintr catches.

Chapter 10

Working with Big Data

by Kunal Mishra and Jade Benjamin-Chung

10.1 The data.table package

It may also be the case that you're working with very large datasets. Generally I would define this as 10+ million rows. As is outlined in this document, the 3 main players in the data analysis space are Base R, `Tidvyerse` (more specifically, `dplyr`), and `data.table`. For a majority of things, Base R is inferior to both `dplyr` and `data.table`, with concise but less clear syntax and less speed. `Dplyr` is architected for medium and smaller data, and while its very fast for everyday usage, it trades off maximum performance for ease of use and syntax compared to `data.table`. An overview of the `dplyr` vs `data.table` debate can be found in this stackoverflow post and all 3 answers are worth a read.

You can also achieve a performance boost by running `dplyr` commands on `data.table`s, which I find to be the best of both worlds, given that a `data.table` is a special type of `data.frame` and fairly easy to convert with the `as.data.table()` function. The speedup is due to `dplyr`'s use of the `data.table` backend and in the future this coupling should become even more natural.

If you want to test whether using a certain coding approach increases speed, consider the `tictoc` package. Run `tic()` before a code chunk and `toc()` after to measure the amount of system time it takes to run the chunk. For example, you might use this to decide if you *really* need to switch a code chunk from `dplyr` to `data.table`.

10.2 Using downsampled data

In our studies with very large datasets, we save "downsampled" data that usually includes a 1% random sample stratified by any important variables, such as year or household id. This allows us to efficiently write and test our code without having to load in large, slow datasets that can cause RStudio to freeze. Be very careful to be sure which dataset you are working with and to label results output accordingly.

10.3 Optimal RStudio set up

Using the following settings will help ensure a smooth experience when working with big data. In RStudio, go to the "Tools" menu, then select "Global Options". Under "General":

Workspace

- **Uncheck** Restore RData into workspace at startup
- Save workspace to RData on exit – choose **never**

History

- **Uncheck** Always save history

Unfortunately RStudio often gets slow and/or freezes after hours working with big datasets. Sometimes it is much more efficient to just use Terminal / gitbash to run code and make updates in git.

Chapter 11

Tidy evaluation and programming with dplyr

by Anna Nguyen, Jade Benjamin-Chung, and Gabby Barratt Heitmann

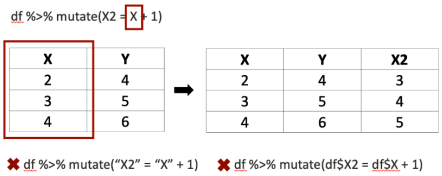
11.1 General Overview

This chapter covers tidy evaluation, a programming pattern in R in which columns are treated as distinct objects within their dataframe's environment. In our lab, data masking most frequently comes up when writing wrapper functions where arguments to indicate column names are supplied as strings. We often do this when we repeat the same code on multiple columns, and want to apply a function to a vector of strings that correspond to column names in a dataframe. For example, we might want to clean multiple columns using the same function or estimate the same model under different feature sets. Here, we try to break down what data masking is, why this error comes up, and common approaches to solve this problem.

Note: the tidyverse uses the term 'data masking' to describe this behavior. This is unrelated to masking in the sense of blinding analysts to treatment assignment, which is discussed in the Reproducibility chapter.

11.1.1 What is Tidy Evaluation?

Within certain tidyverse operations, columns are called as if they were variables. For example, while running `df %>% mutate(X = …)` R recognizes that `X` specifically references a column in `df` without explicitly stating its membership `df %>% mutate(df$X = …)` or calling the column name as a string `df %>% mutate("X" = …)`.



However, this behavior may introduce errors when we attempt to incorporate variables from the global environment within these tidyverse pipelines. In the example above, `column_name = "X"` followed by `df %>% mutate(X2 = column_name + 1)` would yield an error, since `column_name` is not a column in `df` and the variable `column_name` is not defined within the environment of `df`

11.1.2 Using tidy evaluation for data masking

In dplyr-based R programming, we make use of tidy evaluation. This allows us to avoid using base R syntax to reference specific columns in a data frame. By leveraging Tidy evaluation-based data masking, we can employ long pipes with several dplyr verbs to manipulate our data using stand-alone variables that store column names as strings.

For example, consider a data frame "df" that contains a column called "heavyrain" that we want to manipulate. Suppose we wanted to convert the values of "heavyrain" into a factor.

Using base R, which does not mask data, heavyrain must have quotes to be treated as a data-variable:

In a dplyr pipe, heavyrain is being masked using tidy evaluation and will be correctly interpreted as a column because it is recognized as a data-variable: df %>% mutate(outcome = as.factor(heavyrain))

With modified data masking, heavyrain is a string that is coerced into being recognized as a data-variable:

While cleaner and often more convenient, the data frame that var_name is in is now "masked" and we refer to the vectors in the dataframe (data-variables) as though it is an object of its own (an environmental-variable). This is why we can just say the variable's name in the context of a pipe – we treat it as though it's an object defined in our environment. Within normal scripts, this is usually fine, because the data frame is "held on to" in the pipe. However, it can cause some programming hurdles when writing functions that take strings of variable/column names as arguments. In the next section, we briefly describe how to troubleshoot common errors in data masking, as relevant to our lab's work.

11.2 Technical Overview

This section covers the R functions and tools that we often use in the context of data masking, focusing on the bang bang operator (`!!`) with symbol coercion (`sym()`) and the Walrus operator (`:=`).

The combined use of `!!` and `sym()` allows us to use strings, rather than data-variables, to reference column names within dplyr. Together, `!!sym("column_name")` forces dplyr to recognize "column_name" as a data-variable prior to evaluating the rest of the expression, enabling the ability to perform calculations on the column while referring to it as a string. `sym()` is a function that turns strings into symbols. In the context of a dplyr pipe, these symbols are interpreted as data-variables. The `!!` (bang bang) operator tells dplyr to evaluate the sym() expression first, e.g. to unquote its expression (e.g. "column_name") and evaluate it as a pre-existing object, first. This is helpful because often we use `sym("column_name")` within a larger expression, and dplyr might evaluate other elements of the expression first without !!, causing errors.

When we want to create a new column (via mutate or summarize), the Walrus operator (`:=`) allows us to specify the new column's name using a string. For example, while `df %>% mutate("new_column" = values)` would yield an error, `df %>% mutate("new_column" := values)` will correctly create a new column called "new_column". If we want to use a variable representing a string, we can use !! to force the variable to be evaluated before using `:=` to assign the value of the new column.

11.2.1 Example

Suppose we want to write a function "generate_descriptive_table" to summarize how the prevalence of "outcome" varies under different levels of a "risk_factor" in a data frame "df"

We can start by writing the function shell:

Next, we can filter the data frame for only rows in which "rf" and "outcome" are not missing. We can use !! and sym() within filter to evaluate the strings stored in "rf" and "outcome". Note that defining `!!sym(outcome)` or `!!sym(outcome)` in variables *outside of the dplyr pipeline* will *not* work.

Similarly, we use !! and sym() in group_by to evaluate column name, stored as a string in the argument "rf"

Finally, we can use the walrus operator, !! and sym() with "summarize" to create a new column that takes the mean of the column referenced in "rf". We also use "glue" or "paste" to give the new column an informative name that includes the "outcome" it describes.

OR

OR

Chapter 12

Data validation and unit testing

by Jade Benjamin-Chung

In epidemiologic analyses, mistakes in data processing can silently change results without producing any error. A merge with the wrong key can duplicate or drop observations; a filter with the wrong condition can exclude the wrong subgroup; a variable coded as character instead of numeric can cause a model to run but produce nonsensical estimates. These problems are often invisible in the final output — the code runs, a table is produced, and the numbers look plausible enough to go unquestioned. It is best to catch these errors early and systematically. The core idea is simple: at every step where data could go wrong, write a check that will fail and stop the code from proceeding if something is not as expected.

This chapter covers two complementary practices: data validation with `assert_that()`, which checks that your real data meets expectations at each step of an analysis script, and unit testing with `test_that()`, which verifies that your custom functions produce correct output using small synthetic datasets.

12.1 Data validation with assertions in analysis scripts

An assertion is a statement that something must be true at a particular point in your code. If the assertion fails, the script stops immediately with an informative error message. This is far preferable to the alternative — silently producing wrong results that you discover weeks later (or never).

We use the `assertthat` package for assertions. Place assertions after any operation that changes the structure or content of your data: merges, filters, reshapes, variable creation, and aggregation. You write code for what you want to check that results in a TRUE or FALSE value, and `assert_that()` will throw an error if the value is FALSE and stop the code from running further.

12.1.1 After merges and joins

Merges are the single most common source of silent errors in our work. When you join two datasets, always check that the result has the number of rows you expect.

Example: Merging household survey data with an individual symptom module. Suppose `household` has one row per household and `symptoms` has one row per individual. We want to add household-level covariates (e.g., floor type, water source) to each individual's record.

The key principle: **you should know what the row count should be before the merge, and verify it after.** A left join of individuals onto households should not change the number of rows. If it does, there are duplicate keys in the household data.

```
library(dplyr)
library(assertthat)

# Load data
household <- readRDS(here::here("data", "household.rds"))
symptoms <- readRDS(here::here("data", "symptoms.rds"))

# Check pre-merge expectations
n_individuals <- nrow(symptoms)
n_households <- n_distinct(household$household_id)

assert_that(
  n_distinct(household$household_id) == nrow(household),
  msg = "household data has duplicate household_id values"
)

assert_that(
  all(symptoms$household_id %in% household$household_id),
  msg = "some individuals have household_ids not found in household data"
)

# Merge
individual_data <- symptoms %>%
  left_join(household, by = "household_id")
```

```
# Check post-merge expectations
assert_that(
  nrow(individual_data) == n_individuals,
  msg = paste0(
    "merge changed number of rows: expected ", n_individuals,
    ", got ", nrow(individual_data),
    ". Check for duplicate household_ids in household data."
  )
)

assert_that(
  sum(is.na(individual_data$floor_type)) == 0,
  msg = "floor_type has missing values after merge - household data may be incomplete"
)
```

12.1.2 After filters and subsets

When you filter data to a subgroup, verify that the resulting sample size is plausible. This example restricts the dataset to children under 5 and checks three things: that the filtered dataset is not empty, that it actually removed some rows, and that the resulting sample size is within a plausible range based on enrollment records.

```
# Restrict to children under 5
children_under5 <- individual_data %>%
  filter(age_years < 5)

assert_that(
  nrow(children_under5) > 0,
  msg = "no children under 5 found - check age variable coding"
)

assert_that(
  nrow(children_under5) < nrow(individual_data),
  msg = "filter did not remove any rows - all individuals are under 5?"
)

# If you know the approximate expected count from enrollment records
assert_that(
  nrow(children_under5) >= 800 & nrow(children_under5) <= 1200,
  msg = paste0(
    "expected ~1000 children under 5 based on enrollment, got ",
    nrow(children_under5)
  )
)
```

12.1.3 After variable creation

When you create or recode variables, check that the results are within expected ranges. This example calculates diarrhea prevalence by village and checks that all prevalence values are between 0 and 1 and that no village has zero individuals.

```
# Calculate prevalence
village_prev <- individual_data %>%
  group_by(village_id) %>%
  summarise(
    n = n(),
    n_cases = sum(has_diarrhea, na.rm = TRUE),
    prevalence = n_cases / n
  )

assert_that(
  all(village_prev$prevalence >= 0 & village_prev$prevalence <= 1),
  msg = "prevalence values outside [0, 1] - check numerator and denominator"
)

assert_that(
  all(village_prev$n >= 1),
  msg = "some villages have 0 individuals - check village_id coding"
)
```

12.1.4 Checking for unexpected missing data

Missing data can appear at any stage. Check for it explicitly after operations that might introduce it. This example creates an age category variable using `case_when()` and checks that no observations were left as NA, which would indicate that some age_years values didn't match any of the specified conditions.

```
# After creating a derived variable
individual_data <- individual_data %>%
  mutate(
    age_category = case_when(
      age_years < 1 ~ "infant",
      age_years < 5 ~ "child",
      age_years < 18 ~ "adolescent",
      age_years >= 18 ~ "adult"
    )
  )

# case_when returns NA if no condition matches - catch that
assert_that(
  sum(is.na(individual_data$age_category)) == 0,
  msg = paste0(
```

```
    "age_category has ", sum(is.na(individual_data$age_category)),
    " NAs - check for missing or unexpected age_years values"
  )
)
```

12.2 Writing a validation function

For checks that you repeat across scripts (e.g., verifying a dataset after loading), write a reusable validation function. This example wraps a set of checks into a reusable function that can be called at the top of any analysis script that loads the merged individual-level dataset, verifying that required columns are present, IDs are unique, household counts are close to expected, and age values are plausible.

```
#' Validate the merged individual-level analysis dataset
#' @param data the merged individual-level dataset
#' @param expected_n_hh expected number of unique households
#' @param expected_n_ind expected number of individuals (approximate)
validate_individual_data <- function(data, expected_n_hh, expected_n_ind) {

  # Check that required columns are present
  required_cols <- c("individual_id", "household_id", "village_id",
                     "age_years", "has_diarrhea", "floor_type")
  missing_cols <- setdiff(required_cols, names(data))
  assert_that(
    length(missing_cols) == 0,
    msg = paste0("missing required columns: ", paste(missing_cols, collapse = ", "))
  )

  # Check for duplicated rows
  assert_that(
    n_distinct(data$individual_id) == nrow(data),
    msg = "individual_id is not unique - dataset has duplicate rows"
  )

  # Check that counts are as expected
  n_hh <- n_distinct(data$household_id)
  assert_that(
    abs(n_hh - expected_n_hh) / expected_n_hh < 0.05,
    msg = paste0(
      "number of households (", n_hh, ") differs from expected (",
      expected_n_hh, ") by more than 5%"
    )
  )
```

```
  # Check that variable levels are within expected ranges
  assert_that(
    all(data$age_years >= 0 & data$age_years < 120, na.rm = TRUE),
    msg = "age_years contains implausible values"
  )

  message("All validation checks passed.")
}
```

You can call this function at the top of each analysis script that loads the merged dataset:

```
individual_data <- readRDS(here::here("data", "individual_data.rds"))
validate_individual_data(
  data = individual_data,
  expected_n_hh = 500,
  expected_n_ind = 2000
)
```

12.3 Unit testing with `testthat`

The assertions described above are embedded in your analysis scripts and run every time the script runs. For custom functions that you use across multiple projects or scripts, more structured testing with the `testthat` package is appropriate.

Unit tests verify that a function produces the correct output for known inputs. They are especially valuable for:

- Functions that compute estimates, standard errors, or confidence intervals
- Data cleaning functions that recode, reshape, or aggregate
- Any function that will be used by multiple people or in multiple scripts

12.3.1 Setting up unit tests

Create a `9-tests/` directory in your project and add test files that mirror your function files:

```
my-project/
  0-config.R
  0-functions/
      0-base-functions.R
      0-plot-functions.R
  1-data-processing/
  2-analysis/
  9-tests/
      test-base-functions.R
      test-plot-functions.R
```

12.3.2 Writing unit tests

Each test file should source the corresponding function file and use `testthat` to verify expected behavior. Unit tests are most relevant for functions that get reused across multiple projects or by multiple people — where a silent change in behavior could propagate widely.

Suppose you have a function called `fit_glm_robust` in `0-base-functions.R` that works correctly. Six months later, you or a student modifies it (adds a new option, changes how it handles NAs, refactors it) and accidentally breaks the original behavior. Unit tests catch that immediately. Each `test_that()` block does three things:

1. Creates a small fake dataset with known values
2. Runs the function on it
3. Checks that the output matches what you expect

You're not testing on your real data — you're testing the function's logic in isolation with inputs you control completely.

This differs from `assert_that()` earlier in this chapter, which checks the data — did the merge produce the right number of rows, are prevalence values between 0 and 1, etc. Those run every time your analysis runs, on your real data. The two practices complement each other: `test_that()` protects your **functions** from being broken by future edits, and `assert_that()` protects your **analysis** from being wrong due to unexpected data.

Functions that wrap regression models are good candidates for unit testing because they combine multiple steps — fitting a model, computing robust standard errors, and formatting output — and a bug in any step can produce results that look plausible but are wrong. For example, a confidence interval computed on the log scale that isn't exponentiated back, or a standard error from the model-based variance instead of the sandwich estimator, will still produce a table with numbers in it. The tests below assume a function called `fit_glm_robust` in `0-base-functions.R` that fits a GLM with a binomial family, computes cluster-robust sandwich standard errors, and returns a one-row data frame with columns for N, point estimate, SE, 95% CI, and p-value.

```
# 9-tests/test-base-functions.R
library(testthat)
source(here::here("0-functions", "0-base-functions.R"))

# ---- fit_glm_robust ----

test_that("fit_glm_robust returns expected columns", {
  set.seed(123)
  test_data <- data.frame(
    Y = rbinom(200, 1, 0.3),
    X = rbinom(200, 1, 0.5),
```

```
    cluster_id = rep(1:50, each = 4)
  )
  result <- fit_glm_robust(
    data = test_data,
    outcome = "Y",
    treatment = "X",
    cluster_var = "cluster_id"
  )
  expected_cols <- c("n", "estimate", "se", "ci_lower", "ci_upper", "pvalue")
  expect_true(all(expected_cols %in% names(result)))
})

test_that("fit_glm_robust CI contains point estimate", {
  set.seed(123)
  test_data <- data.frame(
    Y = rbinom(200, 1, 0.3),
    X = rbinom(200, 1, 0.5),
    cluster_id = rep(1:50, each = 4)
  )
  result <- fit_glm_robust(
    data = test_data,
    outcome = "Y",
    treatment = "X",
    cluster_var = "cluster_id"
  )
  expect_true(result$ci_lower <= result$estimate)
  expect_true(result$ci_upper >= result$estimate)
})

test_that("fit_glm_robust returns correct N", {
  set.seed(123)
  test_data <- data.frame(
    Y = rbinom(200, 1, 0.3),
    X = rbinom(200, 1, 0.5),
    cluster_id = rep(1:50, each = 4)
  )
  result <- fit_glm_robust(
    data = test_data,
    outcome = "Y",
    treatment = "X",
    cluster_var = "cluster_id"
  )
  expect_equal(result$n, 200)
})

test_that("fit_glm_robust detects null effect with synthetic data", {
```

```
  set.seed(456)
  test_data <- data.frame(
    Y = rbinom(500, 1, 0.3),
    X = rbinom(500, 1, 0.5),
    cluster_id = rep(1:100, each = 5)
  )
  result <- fit_glm_robust(
    data = test_data,
    outcome = "Y",
    treatment = "X",
    cluster_var = "cluster_id"
  )
  # With no true effect, CI should include the null
  expect_true(result$ci_lower <= 0 & result$ci_upper >= 0)
})

test_that("fit_glm_robust errors on missing outcome variable", {
  test_data <- data.frame(X = c(0, 1), cluster_id = c(1, 2))
  expect_error(fit_glm_robust(
    data = test_data,
    outcome = "Y",
    treatment = "X",
    cluster_var = "cluster_id"
  ))
})
```

12.3.3 Running tests

You can run all tests from the project root:

```
# Run all tests in the 9-tests/ directory
testthat::test_dir(here::here("9-tests"))

# Run a specific test file
testthat::test_file(here::here("9-tests", "test-base-functions.R"))
```

You can also add a line to your project's master bash script to run tests before the analysis:

```
# In run_all.sh
Rscript -e 'testthat::test_dir("9-tests")' || { echo "Tests failed"; exit 1; }
```

This ensures that if any test fails, the analysis pipeline stops before producing results.

12.3.4 What to test

Not every function needs a full test suite. Prioritize testing:

- **Functions that implement statistical methods** (e.g., custom estimators, bootstrap procedures, variance calculations). These are high-consequence and hard to verify by inspection.
- **Complex data cleaning functions** used across multiple scripts or projects. A bug here propagates everywhere.
- **Functions with complex logic** — many conditionals, edge cases, or interactions between arguments.

For simple helper functions (e.g., formatting a label), inline assertions in the analysis script are usually sufficient.

12.4 Summary of practices

- Place `assertthat::assert_that()` calls after every merge, filter, reshape, and variable creation step in your analysis scripts.
- Always check row counts before and after merges. Know what the correct number should be.
- Check that computed values are within plausible ranges.
- Write reusable validation functions for datasets you load in multiple scripts.
- Use `test_that()` for custom functions that are shared across scripts or projects.
- Add test execution to your master bash script so failures halt the pipeline.

Chapter 13

Github

by Stephanie Djajadi and Nolan Pokpongkiat

13.1 Basics

- A detailed tutorial of Git can be found here on the CS61B website.
- If you are already familiar with Git, you can reference the summary at the end of Section B.
- If you have made a mistake in Git, you can refer to this article to undo, fix, or remove commits in git.

13.2 Github Desktop

While knowing how to use Git on the command line will always be useful since the full power of Git and its customizations and flexibilty is designed for use with the command line, Github also provides Github Desktop as an graphical interface to do basic git commands; you can do all of the basic functions of Git using this desktop app. Feel free to use this as an alternative to Git on the command line if you prefer.

13.3 Git Branching

Branches allow you to keep track of multiple versions of your work simultaneously, and you can easily switch between versions and merge branches together once you've finished working on a section and want it to join the rest of your code. Here are some cases when it may be a good idea to branch:

- You may want to make a dramatic change to your existing code (called refactoring) but it will break other parts of your project. But you want

to be able to simultaneously work on other parts or you are collaborating with others, and you don't want to break the code for them.
- You want to start working on a new part of the project, but you aren't sure yet if your changes will work and make it to the final product.
- You are working with others and don't want to mix up your current work with theirs, even if you want to bring your work together later in the future.

A detailed tutorial on Git Branching can be found here. You can also find instructions on how to handle merge conflicts when joining branches together.

13.4 Example Workflow

A standard workflow when starting on a new project and contributing code looks like this:

Command	Description
SETUP: FIRST TIME ONLY: `git clone <url> <directory_name>`	Clone the repo. This copies of all the project files in its current state on Github to your local computer.
1. `git pull origin master`	update the state of your files to match the most current version on GitHub
2. `git checkout -b <new_branch_name>`	create new branch that you'll be working on and go to it
3. Make some file changes	work on your feature/implementation
4. `git add -p`	add changes to stage for commit, going through changes line by line
5. `git commit -m <commit message>`	commit files with a message
6. `git push -u origin <branch_name>`	push branch to remote and set to track (-u only needed if this is first push)
7. Repeat step 4-5.	work and commit often
8. `git push`	push work to remote branch for others to view
9. Follow the link given from the `git push` command to submit a pull request (PR) on GitHub online	PR merges in work from your branch into master
(10.) Your changes and PR get approved, your reviewer deletes your remote branch upon merging	
11. `git fetch --all --prune`	clean up your local git by untracking deleted remote branches

Other helpful commands are listed below.

13.5 Commonly Used Git Commands

Command	Description
`git clone <url> <directory_name>`	clone a repository, only needs to be done the first time
`git pull origin master`	pull from `master` before making any changes
`git branch`	check what branch you are on
`git branch -a`	check what branch you are on + all remote branches
`git checkout -b <new_branch_name>`	create new branch and go to it (only necessary when you create a new branch)
`git checkout <branch name>`	switch to branch
`git add <file name>`	add file to stage for commit
`git add -p`	adds changes to commit, showing you changes one by one
`git commit -m <commit message>`	commit file with a message
`git push -u origin <branch_name>`	push branch to remote and set to track (-u only works if this is first push)
`git branch --set-upstream-to origin <branch_name>`	set upstream to `origin/<branch_name>` (use if you forgot -u on first push)
`git push origin <branch_name>`	push work to branch
`git checkout <branch_name> git merge master`	switch to branch and merge changes from `master` into `<branch_name>` (two commands)
`git merge <branch_name> master`	switch to branch and merge changes from `master` into `<branch_name>` (one command)
`git checkout --track origin/<branch_name>`	pulls a remote branch and creates a local branch to track it (use when trying to pull someone else's branch onto your local computer)
`git push --delete <remote_name> <branch_name>`	delete remote branch
`git branch -d <branch_name>`	deletes local branch, -D to force
`git fetch --all --prune`	untrack deleted remote branches

13.6 How often should I commit?

It is good practice to commit every 15 minutes, or every time you make a significant change. It is better to commit more rather than less.

13.7 What should be pushed to Github?

Never push .Rout files! If someone else runs an R script and creates an .Rout file at the same time and both of you try to push to github, it is incredibly difficult to reconcile these two logs. If you run logs, keep them on your own system or (preferably) set up a shared directory where all logs are name and date timestamped.

There is a standardized `.gitignore` for `R` which you can download and add to your project. This ensures you're not committing log files or things that would otherwise best be left ignored to GitHub. This is a great discussion of project-oriented workflows, extolling the virtues of a self-contained, portable projects, for your reference.

Chapter 14

Unix commands

by Stephanie Djajadi, Kunal Mishra, Anna Nguyen, and Jade Benjamin-Chung

We typically use Unix commands in Terminal (for Mac users) or Git Bash (for Windows users) to

1. Run a series of scripts in parallel or in a specific order to reproduce our work
2. To check on the progress of a batch of jobs
3. To use git and push to github

14.1 Basics

On the computer, there is a desktop with two folders, `folder1` and `folder2`, and a file called `file1.` Inside `folder1`, we have a file called `file2.` Mac users can run these commands on their terminal; it is recommended that Windows users use Git Bash, not Windows PowerShell.

14.2 Syntax for both Mac/Windows

When typing in directories or file names, quotes are necessary if the name includes spaces.

Command	Description
`cd desktop/folder1`	Change directory to `folder1`
`pwd`	Print working directory
`ls`	List files in the directory
`cp "file2" "newfile2"`	Copy file (remember to include file extensions when typing in file names like `.pdf` or `.R`)

Command	Description
`mv "newfile2" "file3"`	Rename `newfile2` to `file3`
`cd ..`	Go to parent of the working directory (in this case, `desktop`)
`mv "file1" folder2`	Move `file1` to `folder2`
`mkdir folder3`	Make a new folder in `folder2`
`rm <filename>`	Remove files
`rm -rf folder3`	Remove directories (`-r` will attempt to remove the directory recursively, `-rf` will force removal of the directory)
`clear`	Clear terminal screen of all previous commands

14.3 Running Bash Scripts

Windows	Mac / Linux	Description
`chmod +750 <filename.sh>`	`chmod +x <filename.sh>`	Change access permissions for a file (only needs to be done once)
`./<filename.sh>`	`./<filename.sh>`	Run file (`./` to run any executable file)
`bash bash_script_name.sh &`	`bash bash_script_name.sh &`	Run shell script in the background

14.4 Running Rscripts in Windows

Note: This code seems to work only with Windows Command Prompt, not with Git Bash.

When R is installed, it comes with a utility called Rscript. This allows you to run R commands from the command line. If Rscript is in your `PATH,` then typing Rscript into the command line, and pressing enter, will not error. Otherwise, to use Rscript, you will either need to add it to your PATH (as an environment variable), or append the full directory of the location of Rscript on your machine. To find the full directory, search for where R is installed your computer. For instance, it may be something like below (this will vary depending on what version of R you have installed):

```
C:\Program Files\R\R-3.6.0\bin
```

For appending the `PATH` variable, please view this link. I strongly recommend completing this option.

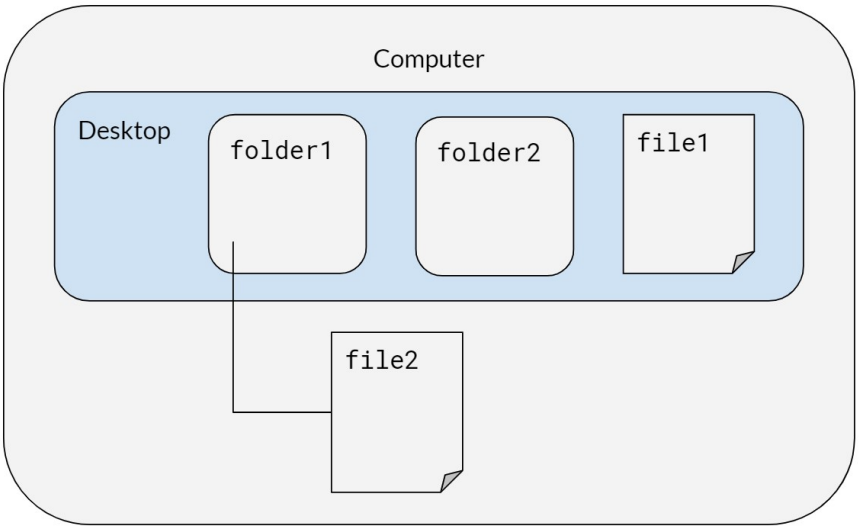


Figure 14.1: Here is our example desktop.

If you add the PATH as an environment variable, then you can run this line of code to test: `Rscript -e "cat('this is a test')"`, where the `-e` flag refers to the expression that will be executed.

If you do not add the PATH as an environment variable, then you can run this line of code to replicate the results from above: `"C:\Program Files\R\R-3.6.0\bin" -e "cat('this is a test')"`

To run an R script from the command line, we can say: `Rscript -e "source('C:/path/to/script/some_code.R')"`

14.4.1 Common Mistakes

- Remember to include all of the quotation marks around file paths that have a spaces.
- If you attempt to run an R script but run into `Error: '\U' used without hex digits in character string starting "'C:\U"`, try replacing all `\` with `\\` or `/`.

14.5 Checking tasks and killing jobs

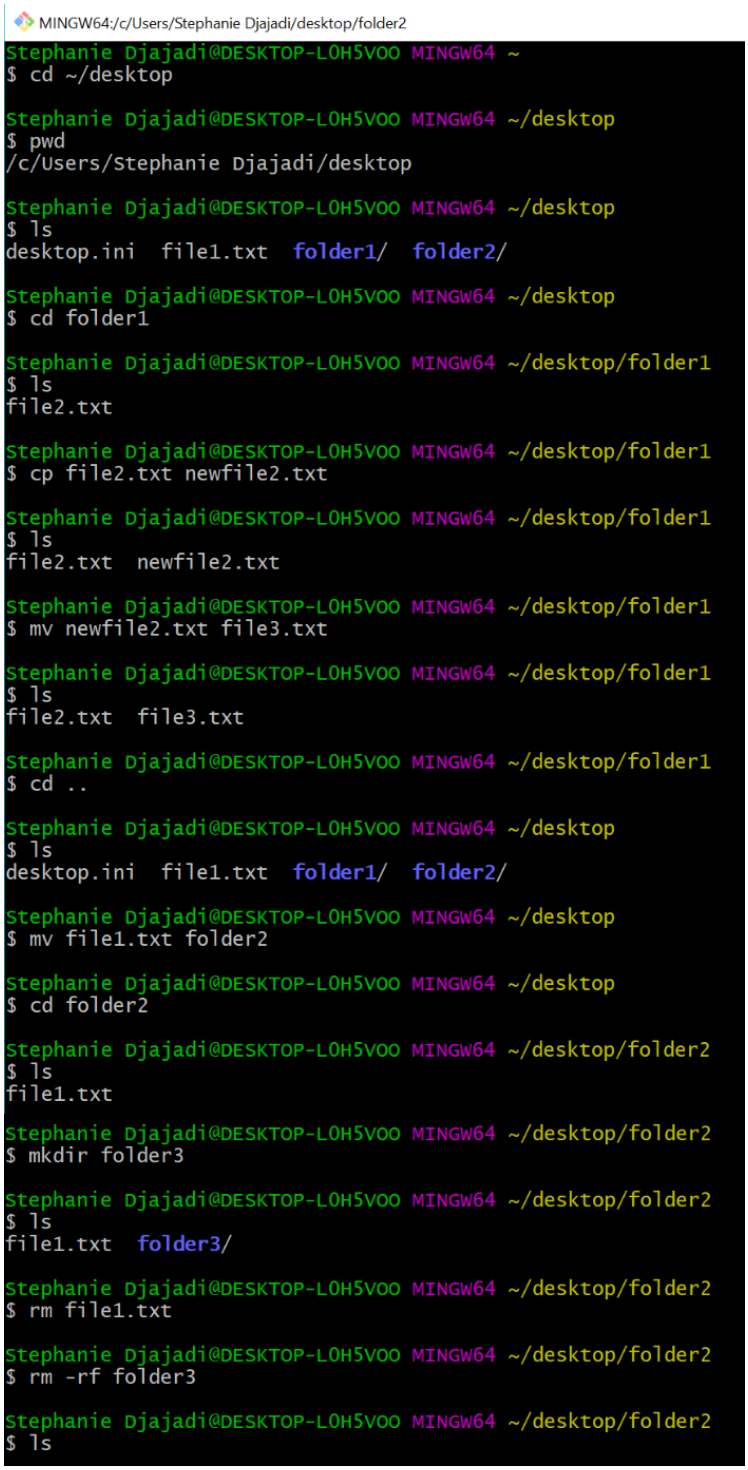


Figure 14.2: Here is an example of what your terminal might look like after executing the commands in the order listed above.

Windows	Mac / Linux	Description
`tasklist`	`ps -v`	List all processes on the command line
	`top -o [cpu/rsize]`	List all running processes, sorted by CPU or memory usage
`taskkill /F /PID pid_number`	`kill <PID_number>`	Kill a process by its process ID
`taskkill /IM "process name" /F`		Kill a process by its name
`start /b program.exe`		Runs jobs in the background (exclude `/b` if you want the program to run in a new console)
	`nohup`	Prevents jobs from stopping
	`disown`	Keeps jobs running in the background even if you close R
`taskkill /?`		Help, lists out other commands

To kill a task in Windows, you can also go to Task Manager > More details > Select your desired app > Click on End Task.

14.6 Running big jobs

For big data workflows, the concept of "backgrounding" a bash script allows you to start a "job" (i.e. run the script) and leave it overnight to run. At the top level, a bash script (`0-run-project.sh`) that simply calls the directory-level bash scripts (i.e. `0-prep-data.sh`, `0-run-analysis.sh`, `0-run-figures.sh`, etc.) is a powerful tool to rerun every script in your project. See the included example bash scripts for more details.

- **Running Bash Scripts in Background**: Running a long bash script is not trivial. Normally you would run a bash script by opening a terminal and typing something like `./run-project.sh`. But what if you leave your computer, log out of your server, or close the terminal? Normally, the bash script will exit and fail to complete. To run it in background, type `./run-project.sh &; disown`. You can see the job running (and CPU utilization) with the command `top` or `ps -v` and check your memory with `free -h`.

Alternatively, to keep code running in the background even when an SSH connection is broken, you can use `tmux`. In terminal or gitbash follow the steps below. This site has useful tips on using `tmux`.

- **Deleting Previously Computed Results**: One helpful lesson we've learned is that your bash scripts should remove previous results (computed and saved by scripts run at a previous time) so that you never mix results from one run with a previous run. This can happen when an R script errors out before saving its result, and can be difficult to catch because your previously saved result exists (leading you to believe everything ran correctly).
- **Ensuring Things Ran Correctly**: You should check the `.Rout` files generated by the R scripts run by your bash scripts for errors once things are run. A utility file is include in this repository, called `runFileSaveLogs`, and is used by the example bash scripts to… run files and save the generated logs. It is an awesome utility and one I definitely recommend using. Before using `runFileSaveLogs`, it is necessary to put the file in the home working directory. For help and documentation, you can use the command `./runFileSaveLogs -h`. See example code and example usage for `runFileSaveLogs` below.

14.6.1 Example code for `runfileSaveLogs`

14.6.2 Example usage for `runfileSaveLogs`

This example bash script runs files and generates logs for five scripts in the `kaiserflu/3-figures` folder. Note that the `-i` flag is used as an identifier to add `figures` to the filename of each log.

Chapter 15

Reproducible Environments

by Anna Nguyen and Jade Benjamin-Chung

15.1 Package Version Control with renv

15.1.1 Introduction

Replicable code should produce the same results, regardless of when or where it's run. However, our analyses often leverage open-source R packages that are developed by other teams. These packages continue to be developed after research projects are completed, which may include changes to analysis functions that could impact how code runs for both other team members and external replicators.

For example, suppose we had used a function that took in one argument, such that our code contained `example_function(arg_a = "a")`. A few months after we publish our code, the package developers update the function to take in another mandatory argument `arg_b`. If someone runs our code, but has the most recent version of the package, they'll receive an error message that the argument `arg_b` is missing and will not be able to full reproduce our results.

To ensure that the right functions are used in replication efforts, it is important for us to keep track of package versions used in each project.

`renv` can be to promote reproducible environments within R projects. `renv` creates individual package libraries for each project instead of having all projects, which may use different versions of the same package, share the same package library. However, for projects that use many packages, this process can be memory intensive and increase the time needed for a new users to start running code.

In this lab manual chapter, we provide a quick tutorial for integrating `renv`

into research workflows. For more detailed instructions, please refer to the `renv` package vignette.

15.1.2 Implementing renv in projects

Ideally, `renv` should be initiated at the start of projects and updated continuously when new packages are introduced in the codebase. However, this process can be initated at any point in a project

To add `renv` to your workflow, follow these steps:

1. Install the `renv` package by running `install.packages("renv")`
2. Create an RProject file and ensure that your working directory is set to the correct folder
3. In the R console, run `renv::init()` to intiialize renv in your R Project
4. This will create the following files: `renv.lock`, `.Rprofile`, `renv/settings.json` and `renv/activate.R`. Commit and push these files to GitHub so that they're accessible to other users.
5. As you write code, update the project's R library by running `renv::snapshot()` in the R console
6. Add `renv::restore()` to the head of your config file, to make sure that all users that run your code are on the same package versions.

15.1.3 Using projects with renv

If you're starting to work on an ongoing project that already has `renv` set up, follow these steps to ensure that you're using the same project versions.

1. Install the `renv` package by running `install.packages("renv")`
2. Pull the most updated version of the project from GitHub
3. Open the project's RProject file
4. Run `renv::restore()`. In our lab's projects, this is often already found at the top of the config file, so you can just run scripts as is.
5. This will pull up a list of the project's packages that need to be updated for you to be consistent with the project. The console will ask if you want to proceed with updating these packages - type "Y" to continue.
6. Wait for the correct versions of each package to install/update. This may take some time, depending on how many packages the project uses.
7. Your R environment should now be using the same package versions as specified in the `renv` lock file. You should now be able to replicate the code.
8. If you make edits to the code and introduce new/updated packages, see the section above for instructions on how to make updates.

15.2 Documenting the R version

`renv` tracks package versions but does not track the version of R itself. Since R updates can change behavior (e.g., changes to the default `stringsAsFactors` argument in R 4.0, or updates to the random number generator in R 3.6), it is important to document the R version used in each project.

Add the following to your project's README: `This project was developed using R version 4.3.1 (2023-06-16). Package versions are tracked with renv (see renv.lock).`

You can find your current R version by running `R.version.string` in the console. Update this line in the README whenever you upgrade R for a project.

For published code, also include the R version in the `renv.lock` file header, which `renv` does automatically. Reviewers or replicators can then check whether their R version matches before attempting to reproduce results.

15.3 Ensuring consistency between local and cluster environments

If you develop code locally and run it on Sherlock, differences between your local R environment and the cluster environment can cause code to behave differently or fail. The most common sources of discrepancy are the R version, package versions, and system libraries.

Use the same R version on both. Check which R versions are available on Sherlock by running `module avail R` on the command line. Load a specific version with `module load R/4.3.1` (replacing with your version). If your local R version is not available on Sherlock, either update your local installation to match an available Sherlock version, or vice versa. Document the version you are using in your README.

Use `renv` on Sherlock. The same `renv.lock` file that tracks packages locally should be used on Sherlock. After cloning your repo on Sherlock, run `renv::restore()` to install the correct package versions. Note that some packages with compiled code (e.g., `data.table`, `Stan`-based packages) may need to be recompiled on Sherlock due to differences in the operating system and system libraries. If `renv::restore()` fails for specific packages, try installing them manually with `renv::install("package_name")` and then running `renv::snapshot()` to update the lock file.

Test on Sherlock before running full analyses. Before submitting large batch jobs, run a small test script interactively on a development node (`sdev`) to verify that packages load correctly and that a simple version of your analysis produces expected output. This catches environment mismatches before you spend allocation hours on jobs that will fail.

Chapter 16

Code Publication

by Nolan Pokpongkiat

16.1 Checklist overview

1. Fill out file headers
2. Clean up comments
3. Document functions
4. Remove deprecated filepaths
5. Ensure project runs via bash
6. Complete the README
7. Clean up feature branches
8. Create Github release

16.2 Fill out file headers

Every file in a project should have a header that allows it to be interpreted on its own. It should include the name of the project and a short description for what this file (among the many in your project) does specifically. See template here.

16.3 Clean up comments

Make sure comments in the code are for code documentation purposes only. Do not leave comments to self in the final script files.

16.4 Document functions

Every function you write must include a header to document its purpose, inputs, and outputs. See template for the function documentation here.

16.5 Remove deprecated filepaths

All file paths should be defined in 0-config.R, and should be set relative to the project working directory. All absolute file paths from your local computer should be removed, and replaced with a relative path. If a third party were to re-run this analysis, if they need to download data from a separate source and change a filepath in the 0-config.R to match, make sure to specify in the README which line of 0-config.R needs to be substituted.

16.6 Ensure project runs via bash

The project should be configured to be entirely reproducible by running a master bash script, run-project.sh, which should live at the top directory. This bash script can call other bash scripts in subfolders, if necessary. Bash scripts should use the runFileSaveLogs utility script, which is a wrapper around the Rscript command, allowing you to specify where .Rout log files are moved after the R scripts are run.

See usage and documentation here.

16.7 Complete the README

A README.md should live at the top directory of the project. This usually includes a Project Overview and a Directory Structure, along with the names of the contributors and the Creative Commons License. See below for a template:

> **Overview**
>
> To date, coronavirus testing in the US has been extremely limited. Confirmed COVID-19 case counts underestimate the total number of infections in the population. We estimated the total COVID-19 infections – both symptomatic and asymptomatic – in the US in March 2020. We used a semi-Bayesian approach to correct for bias due to incomplete testing and imperfect test performance.
>
> **Directory structure**
>
> - 0-config.R: configuration file that sets data directories, sources base functions, and loads required libraries
> - 0-base-functions: folder containing scripts with functions used in the analysis

  - 0-base-functions.R: R script containing general functions used across the analysis
  - 0-bias-corr-functions.R: R script containing functions used in bias correction
  - 0-bias-corr-functions-undertesting.R: R script containing functions used in bias correction to estimate the percentage of underestimation due to incomplete testing vs. imperfect test accuracy
  - 0-prior-functions.R: R script containing functions to generate priors
- 1-data: folder containing data processing scripts NOTE: some scripts are deprecated
- 2-analysis: folder containing analysis scripts. To rerun all scripts in this subdirectory, run the bash script 0-run-analysis.sh.
  - 1-obtain-priors-state.R: obtain priors for each state
  - 2-est-expected-cases-state.R: estimate expected cases in each state
  - 3-est-expected-cases-state-perf-testing.R: estimate expected cases in each state, estimate the percentage of underestimation due to incomplete testing vs. imperfect test accuracy
  - 4-obtain-testing-protocols.R: find testing protocols for each state.
  - 5-summarize-results.R: summarize results; obtain results for in text numerical results.
- 3-figure-table-scripts: folder containing figure scripts. To rerun all scripts in this subdirectory, run the bash script 0-run-figs.sh.
  - 1-fig-testing.R: creates plot of testing patterns by state over time
  - 2-fig-cases-usa-state-bar.R: creates bar plot of confirmed vs. estimated infections by state
  - 3a-fig-map-usa-state.R: creates map of confirmed vs. estimated infections by state
  - 3b-fig-map-usa-state-shiny.R: creates map of confirmed vs. estimated infections by state with search functionality by state
  - 4-fig-priors.R: creates figure with priors for US as a whole

  - 5-fig-density-usa.R: creates figure of distribution of estimated cases in the US
  - 6-table-data-quality.R: creates table of data quality grading from COVID Tracking Project
  - 7-fig-testpos.R: creates figure of the probability of testing positive among those tested by state
  - 8-fig-percent-undertesting-state.R: creates figure of the percentage of under estimation due to incomplete testing
- 4-figures: folder containing figure files.
- 5-results: folder containing analysis results objects.
- 6-sensitivity: folder containing scripts to run the sensitivity analyses

Contributors: Jade Benjamin-Chung, Sean L. Wu, Anna Nguyen, Stephanie Djajadi, Nolan N. Pokpongkiat, Anmol Seth, Andrew Mertens

Wu SL, Mertens A, Crider YS, Nguyen A, Pokpongkiat NN, Djajadi S, et al. Substantial underestimation of SARS-CoV-2 infection in the United States due to incomplete testing and imperfect test accuracy. medRxiv. 2020; 2020.05.12.20091744. doi:10.1101/2020.05.12.20091744

When possible, also include a description of the RDS results that are generated, detailing what data sources were used, where the script lives that creates it, and what information the RDS results hold.

16.8 Clean up feature branches

In the remote repository on Github, all feature branches aside from master should be merged in and deleted. All outstanding PRs should be closed.

16.9 Create Github release

Once all of these items are verified, create a tag to make a Github release, which will tag the repository, creating a marker at this specific point in time.

Detailed instructions here.

Chapter 17

Data Publication

Adapted from Fanice Nyatigo and Ben Arnold's chapter in the Proctor-UCSF Lab Manual

17.1 Overview

Warning! ***NEVER*** *push a dataset into the public domain (e.g., GitHub, OSF) without first checking with Jade to ensure that it is appropriately de-identified and we have approval from the sponsor and/or human subjects review board to do so. For example, we will need to re-code participant IDs (even if they contain no identifying information) before making data public to completely break the link between IDs and identifiable information stored on our servers.*

If you are releasing data into the public domain, then consider making available *at minimum* a `.csv` file and a codebook of the same name (note: you should have a codebook for internal data as well). We often also make available `.rds` files as well. For example, your `mystudy/data/public` directory could include three files for a single dataset, two with the actual data in `.rds` and `.csv` formats, and a third that describes their contents:

```
analysis_data_public.csv
analysis_data_public.rds
analysis_data_public_codebook.txt
```

In general, datasets are usually too big to save on GitHub, but occasionally they are small. Here is an example of where we actually pushed the data directly to GitHub: https://github.com/ben-arnold/enterics-seroepi/tree/master/data .

If the data are bigger, then maintaining them under version control in your git repository can be unweildy. Instead, we recommend using another stable repository that has version control, such as the Open Science Framework (osf.io). For example, all of the data from the WASH Benefits trials (led by investigators at Berkeley, icddr,b, IPA-Kenya and others) are all stored through data components nested within in OSF projects: https://osf.io/tprw2/. Another good option is Dryad (datadryad.org) or the (Stanford Digital Repository)[https://sdr.stanford.edu/].

We recommend cross-linking public files in GitHub (scripts/notebooks only) and OSF/Dryad/Stanford Digital Repository.

Below are the main steps to making data public, after finalizing the analysis datasets and scripts:
1. Remove Protected Health Information (PHI)
2. Create public IDs or join already created public IDs to the data
3. Create an OSF repository and/or Dryad/Stanford Digital Repository
4. Edit analysis scripts to run using the public datasets and test (optional)
5. Create a public github page for analysis scripts and link to OSF and/or Dryad/Zenodo
6. Go live

17.2 Removing PHI

Once the data is finalized for analysis, the first step is to strip it of Protected Health Information (PHI), or any other data that could be used to link back to specific participants, such as names, birth dates, or GPS coordinates at the village/neighborhood level or below. PHI includes, but is not limited to:

17.2.1 Personal information

These are identifiers that directly point to specific individuals, such as:
- Names, addresses, photographs, date of birth
- A combination of age, sex, and geographic location (below population 20,000) is considered identifiable

17.2.2 Dates

Any specific dates (e.g., study visit dates, birth dates, treatment dates) are usually problematic.
- If a dataset requires high resolution temporal information, coarsen visit or measurement dates to be two variables: year and week of the year (1-52).
- If a dataset requires age, provide that information without a birth date (typically month resolution is sufficient)

> ***Caution!*** *If making changes to the format of dates or ages, make sure your analysis code runs on these modified versions of the data (step 3)!*

17.2.3 Geographic information

Do not include GPS coordinates (longitude, latitude) except in special circumstances where they have been obfuscated/shifted. Reach out to Jade before doing this because it can be complicated.

Do not include place names or codes (e.g., US Zip Codes) if the place contains <20,000 people. For villages or neighborhoods, code them with uninformative IDs. For sub-districts or districts, names are fine.

If an analysis requires GPS locations (e.g., to make a map), then typically we include a disclaimer in the article's data availability statement that explains we cannot make GPS locations public to protect participant confidentiality. As a middle ground, we typically make our *code* public that runs on the geo-located data for transparency, even if independent researchers can't actually run that code (although please be careful to ensure the code itself does not in any way include geographic identifiers).

For more examples of what constitutes PHI, please refer to this link: https://cphs.berkeley.edu/hipaa/hipaa18.html

17.3 Create public IDs

17.3.1 Rationale

The Stanford IRB requires that public datasets not include the original study IDs to identify participants or other units in the study (such as village IDs). The reason is that those IDs are linked in our private datasets to PHI. By creating a new set of public IDs, the public dataset is one step further removed from the potential to link to PHI.

17.3.2 A single set of public IDs for each study

For each study, it is ideal to create a single set of public IDs whenever possible. We could create a new set of public IDs for every public dataset, but the downside is that independent researchers could no longer link data that might be related. By creating a single set of public IDs associated with each internal study ID, public files retain the link.

Maintaining a single set of public IDs requires a shared "bridge" dataset, that includes a row for each study ID and has the associated public ID. For studies

with multiple levels of ID, we would typically have separate bridge datasets for each type of ID (e.g,. cluster ID, participant ID, etc.)

Create a public ID that can be used to uniquely identify participants and that can internally be linked to the original study IDs. We recommend creating a subdirectory in the study's shared data directory to store the public IDs. The shared location enables multiple projects to use the same IDs. Create the IDs using a script that reads in the study IDs, creates a unique (uninformative) public ID for the study IDs, and then saves the bridge dataset. The script should be saved in the same directory as the public ID files.

Caution! *Note that small differences may arise if the new public IDs do not necessarily order participants in the same way as the internal IDs. The small differences are all in estimates that rely on resampling, such as Bootstrap CIs, permutation P-values, and TMLE, as the resampling process may lead tp slightly different re-samples. The key here, to ensure the results are consistent irrespective of the dataset used, is simply to not assign public IDs randomly. Use* `rank()` *on the internal ID instead of* `row_number()` *to ensure that the order is always the same.*

17.3.3 Example scripts

We have created a self-contained and reproducible example that you can run and replicate when making data public for your projects. It contains the following files and folders:

1. `data/final/`- folder containing the projects final data in both csv and rds formats

2. `code/DEMO_generate_public_IDs.R`- creates randomly generated public IDs that can be matched to the trial's assigned patient IDs.

3. `data/make_public/DEMO_internal_to_publicID.csv`- the output from step #2, a bridge dataset with two variables- the new public ID and the patient's assigned ID.

4. `code/DEMO_create_public_datasets.R`- joins the public IDs to the trial's full dataset, and strips it of the assigned patient ID.
5. `data/public/`- folder containing the output from step #3- de-identified public dataset, in csv and rds formats, with uniquely identifying public IDs that cannot be easily linked back to the patient's ID.

The example workflow is accessible via GitHub: https://github.com/proctor-ucsf/dcc-handbook/tree/master/templates/making-data-public

17.4 Create a data repository

First, ensure that you create a codebook and metadata file for each public dataset See the DCC guide on Documenting datasets. Use the same name as the datasets, but with "-codebook.txt" / "-codebook.html" / "-codebook.csv" at the end (depending on the file format for the codebook). One nice option is the R codebook package, which also generates JSON output that is machine-readable.

17.4.1 Steps for creating an Open Science Framework (OSF) repository:

1. Create a new OSF project per these instructions: https://help.osf.io/article/252-create-a-project
2. Create a data component and upload the datasets in .csv and .rds format along with the codebooks. The primary format for public dissemination is .csv but we make the .rds files available too as auxiliary files for convenience.
3. Create a notebook component and upload the final .html files (which will not be on github… but see optional item below)
4. On the OSF landing Wiki, provide some context. Here is a recent example: https://osf.io/954bt/
5. Create a Digital Object Identifier (DOI) for the repository. A DOI is a unique identifier that provides a persistent link to content, such as a dataset in this case. Learn more about DOIs
6. Optional: Complete the software checklist and system requirement guide for the analysis to guide others. Include it on the GitHub README for the project: https://github.com/proctor-ucsf/mordor-antibody

17.5 Edit and test analysis scripts

Make minor changes to the analysis scripts so that they run on public data. If using version control in GitHub, the most straight-forward way is to create a branch from the main git branch that reads in the public files, and then renames the new public ID variable, e.g., "id_public" to the internally recognized ID variable name, e.g. "recordID", when reading in the public data. Re-run all the analysis scripts to ensure that they still work with the public version of the dataset.

17.6 Create a public GitHub page for public scripts

At minimum, we should include all of the scripts required to run the analyses. **IMPORTANT**: ensure you have taken a snapshot and saved your computing

environment using the `renv` package (`renv`).

See examples:
- ACTION - https://github.com/proctor-ucsf/ACTION-public
- NAITRE - https://github.com/proctor-ucsf/NAITRE-primary

> ***Caution!*** *Read through the scripts carefully to ensure there is no PHI in the code itself*

Once a public GitHub page exists, you can create a new component on an OSF project (step 3, above) and link it to the public version of the GitHub repo.

17.7 Go live

On GitHub, it is useful to create an official "release" version to freeze the repository, where you can have "associated files" with each version. Include the .html notebook output as additional files — since they aren't tracked in GitHub, it does provide a way of freezing / saving the HTML output for us and others. OSF examples of a studies from UCSF's Proctor Foundation:
- ACTION - https://osf.io/ca3pe/
- NAITRE - https://osf.io/ujeyb/
- MORDOR Niger antibody study - https://osf.io/dgsq3/

Further reading on end-to-end data management: How to Store and Manage Your Data - PLOS

Chapter 18

Slurm and cluster computing

by Anna Nguyen, Jade Benjamin-Chung, and Gabby Barratt Heitmann

When you need to run a script that requires a large amount of RAM, large files, or that uses parallelization, you can use Sherlock, Stanford's computing cluster. Sherlock uses Slurm, an open source, scalable cluster management and job scheduling system for computing clusters. Jade can email Sherlock managers to get you an account. Please refer to the Sherlock user guide to learn about the system and how to use it. Below, we include a few tips specific to how we use Sherlock in our lab.

18.1 Getting started

To access Sherlock, in terminal, log in using the following syntax and replace "USERNAME" with your Stanford alias. You will be prompted to enter your Stanford password (the same one you use for your email and other accounts) and to complete two-factor authentication.

Once you log in, you can view the contents of your home directory in command line by entering `cd $HOME`. You can create subfolders within this directory using the `mkdir` command. For example, you could make a "code" subdirectory and clone a Github repository there using the following code:

18.1.1 One-Time System Set-Up

To keep the install packages consistent across different nodes, you will need to explicitly set the pathway to your R library directory.

Open your `~/.Renviron` file (`vi ~/.Renviron`) and append the following line:

Note: Once you open the file using `vi [file_name]`*, you must press* `i` *(on Mac OS) or* `Insert` *(on Windows) to make edits. After you finish, hit* `Esc` *to exit editing mode and type* `:wq` *to save and close the file.*

Alternatively, run an R script with the following code on Sherlock:

To load packages that run off of C++, you'll need to set the correct compiler options in your R environment.

Open the Makevars file in Sherlock (`vi ~/.R/Makevars`) and append the following lines

Alternatively, create an R script with the following code, and run it on Sherlock:

18.2 Moving files to Sherlock

The `$HOME` directory is a good place to store code and small test files (quota: 15 GB per user). Save large files to the `$SCRATCH` directory (quota: 100 TB per user). You can read more about storage options on Sherlock here. On the `$SCRATCH` directory, files that are not modified after 90 days are automatically deleted. For this reason, it's best to create a bash script that records the file transfer process for a given project. See example code below:

18.3 Installing packages on Sherlock

When you begin working on Sherlock, you will most likely encounter problems with installing packages. To install packages, login to Sherlock on the command line and open a development node using the command `sdev`. Do not attempt to do this in the RStudio Server (see next section), as you will have to re-do it for every new session you open.

There is a package installation file explicitly written for Sherlock that you should run before testing any code and sourcing the configuration file. You should only have to install packages once. Sherlock requires that you specify the repository where the package is downloaded from. You may also need to add an additional argument to install.packages to prevent the packages from locking after installation:

In order for some R packages to work on Sherlock, it is necessary to load specific software modules before running R. These must be loaded in Sherlock each time you want to use the package in R. For example, for spatial and random effects analyses, you may need the modules/packages below. These modules must also be loaded on the command line prior to opening R in order for package installation to work.

Figuring out the issues with some packages will require some trial and error. If you are still encountering problems installing a package, you may have to install other dependencies manually by reading through the error messages. If you try

to install a dependency from CRAN and it isn't working, it may be a module. You can search for it using the `module spider` command:

However, you can also reach out to the Sherlock team for help. You can email them at srcc-support@stanford.edu. They also hold office hours.

18.4 Testing your code

Both of the following ways to test code on Sherlock are recommended for making small changes, such as editing file paths and making sure the packages and source files load. You should write and test the functionality of your script locally, only testing on Sherlock once major bugs are out.

18.4.1 The command line

There are two main ways to explore and test code on Sherlock. The first way is best for users who are comfortable working on the command line and editing code in base R. Even if you are not comfortable yet, this is probably the better way because these commands will transfer between Sherlock and other cluster computers using Slurm.

Typically, you will want to initially test your scripts by initiating a development node using the command `sdev`. This will allocate a small amount of computing resources for 1 hour. You can access R via command line using the following code.

*Note: for collaboration purposes, it's best for everyone to work with one version of R. Check what version is being used for the project you are working on. Some packages only work with some versions of R, so it's best to keep it consistent.

18.4.2 The Sherlock OnDemand Dashboard

The second way to test and edit code is to use the Sherlock OnDemand Dashboard, accessed by typing login.sherlock.stanford.edu into a web browser. You will be prompted to authenticate the way you would for any Stanford website. This is the best way to edit code for people who are not comfortable accessing & editing in base R in a Shell application.

You can test your code via the Rstudio server on Sherlock. To access this, login to the Dashboard, then click on Interactive Apps in the menu bar and choose R Studio Server. Similar to the sdev node, you have to set various parameters for your session. Choose a version of R and set the time – max. 2 hours. You can play with the other configurations, but this is likely unnecessary, as you should not need huge computing power to test small amounts of code. Keep in mind the more computing power you request, the lower priority your request becomes. You will then wait for the resources to become available, and you will be able to click "Launch" when they are (if you don't mess with the CPU or GPU, this

is usually less than 2 minutes). The screen that opens will look very similar to the RStudio on your local.

Do NOT use the RStudio Server's Terminal to install packages, set your R environment, and do everything else needed to configure Sherlock because you will likely need to re-do it for every session/project. It's best to use the Dashboard/RStudio Server if you are more comfortable testing & editing in RStudio rather than through base R in a Shell application.

18.4.3 Filepaths & configuration on Sherlock

In most cases, you will want to test that the file paths work correctly on Sherlock. You will likely need to add code to the configuration file in the project repository that specifies Sherlock-specific file paths. Here is an example:

18.5 Storage & group storage access

18.5.1 Individual storage

There are multiple places to store your files on Sherlock. Each user has their own `$HOME` directory as well as a `$SCRATCH` directory. These are directories that can be accessed via the command line once you've logged in to Sherlock:

You can also navigate to these using the File Explorer on Sherlock OnDemand.

`$HOME` has a volume quota of 15 GB. `$SCRATCH` has a volume quota of 100 TB, but files here get deleted 90 days after their last modification. Thus, use `$SCRACTH` for test files, exploratory analyses, and temporary storage. Use `$HOME` for long-term storage of important files and more finalized analyses.

You can read more about storage options on Sherlock here.

18.5.2 Group storage

The lab also has a `$GROUP_HOME` and a `$GROUP_SCRATCH` to store files for collaborative use. `$GROUP_HOME` has a volume quota of 1 TB and infinite retention time, whereas `$GROUP_SCRATCH` has a volume quota of 100 TB and the same 90-day retention limit. You can access these via the command line or navigate to them using the File Explorer:

However, saving files to group storage can be tricky. You can try using the scp command in the section "Moving files to Sherlock" to see if you have permission to add files to group directories. Read the next section to ensure any directories you create have the right permissions.

18.5.3 Folder permissions

Generally, when we put folders in `$GROUP_HOME` or `$GROUP_SCRATCH`, it is so that we can collaborate on an analysis within the research group, so multiple people need to be able to access the folders. If you create a new folder in `$GROUP_HOME` or `$GROUP_SCRATCH`, please check the folder's permissions to ensure that other group members are able to access its contents. To check the permissions of a folder, navigate to the level above it, and enter `ls -l`. You will see output like this:

Please review this website to learn how to interpret the code on the left side of this output. The website also tells you how to change folder permissions. In order to ensure that all users and group members are able to access a folder's contents, you can use the following command:

18.6 Running big jobs

Once your test scripts run successfully, you can submit an sbatch script for larger jobs. These are text files with a `.sh` suffix. Use a text editor like Sublime to create such a script. Documentation on sbatch options is available here. Here is an example of an sbatch script with the following options:

- `job-name=run_inc`: Job name that will show up in the Sherlock system
- `begin=now`: Requests to start the job as soon as the requested resources are available
- `dependency=singleton`: Jobs can begin after all previously launched jobs with the same name and user have ended.
- `mail-type=ALL`: Receive all types of email notification (e.g., when job starts, fails, ends)
- `cpus-per-task=16`: Request 16 processors per task. The default is one processor per task.
- `mem=64G`: Request 64 GB memory per node.
- `output=00-run_inc_log.out`: Create a log file called `00-run_inc_log.out` that contains information about the Slurm session
- `time=47:59:00`: Set maximum run time to 47 hours and 59 minutes. If you don't include this option, Sherlock will automatically exit scripts after 2 hours of run time.

The file `analysis.out` will contain the log file for the R script `analysis.R`.

To submit this job, save the code in the chunk above in a script called `myjob.sh` and then enter the following command into terminal:

To check on the status of your job, enter the following code into terminal:

Chapter 19

Checklists

by Jade Benjamin-Chung

19.1 Pre-analysis plan checklist

Before registering a pre-analysis plan, verify that it includes all elements described in the Write and register pre-analysis plans section. See the following pre-analysis plans for examples from our lab:

- Trials of water, sanitation, and hygiene interventions
- Observational study of school-located influenza vaccination
- Re-analysis of malaria intervention trial

19.2 Code checklist

- Does the script run without errors?
- Is code self-contained within repo and/or associated Box folder?
- Is all commented out code / remarks removed?
- Does the header accurately describe the process completed in the script?
- Are file paths relative (using `here()`) rather than absolute?
- Are assertions present after merges, filters, and key variable creation steps?
- Do unit tests pass (`testthat::test_dir('9-tests')`)?
- Are random seeds set where relevant?
- Is the script pushed to its github repository?
- Does the code adhere to the coding style guide?
- Are all warnings ignorable? Should any warnings be intentionally suppressed or addressed?

19.3 Manuscript checklist

This is adapted in part from this article.

- Have you completed the relevant reporting checklist, if applicable? (Collection of checklists)
- Are the study results within the manuscript replicable (i.e., if you rerun the code in the study's repository, the tables and figures will be exactly replicated?)
- Is a target journal selected?
- Is the title declarative, in other words, does it state the object/findings rather than suggest them?
- Is the word count of the manuscript close to the target journal's allowance?
- Does the manuscript adhere to the formatting guide of the target journal?
- Does the manuscript use a consistent voice (passive or active – usually active is preferred … pun intended)?
- Is each figure and table (including supplementary material) referenced in the main text?
- Is there a caption for each figure and table (including supplementary material)?

- Are all numbers in the text (sample sizes, point estimates, CIs, p-values) generated programmatically from the analysis output rather than manually typed?

- Are tables/figures and supplementary material numbered in accordance with their appearance in the main text?
- Does the text use past tense if it is reporting research findings or future tense if it is a study protocol?
- Does the text avoid subjective wording (e.g., "interesting", "dramatic")?
- Does the text use minimal abbreviations, and are all abbreviations defined at first use?
- Does the text avoid directionless words? (e.g., instead of writing, 'Precipitation influences disease risk', write, 'Precipitation was associated with increased disease risk').
- Does the text avoid making causal claims that are not supported by the study design? Be careful about the words "effect", "increase", and "decrease", which are often interpreted as causal.
- Does the text avoid describing results with the word "significant", which can easily be confused with statistical significance? (see references on this topic here)
- Have you drafted author contributions? Do they follow the CRediT Taxonomy for author contributions?

19.4 Figure checklist

- Are the x-axis and y-axis labeled?
- Are the axis limits appropriate (e.g., not truncating data or exaggerating differences)?
- If the figure includes panels, is each panel labeled?
- Are there sufficient numerical / text labels and breaks on the x-axis and y-axis?
- Is the font size appropriate (i.e., large enough to read, not so large that it distracts from the data presented in the figure?)
- Are the colors used colorblind friendly? See a colorblind-friendly palette here, a neat palette generator with colorblind options here, and an article on why this matters here
- Are colors/shapes/line types defined in a legend?
- Are the legends and other labels easy to understand with minimal abbreviations?
- If there is overplotting, is transparency used to show overlapping data?
- Are 95% confidence intervals or other measures of precision shown, if applicable?
- Does the figure have a consistent theme/style with other figures in the manuscript?

Chapter 20

Resources

by Jade Benjamin-Chung and Kunal Mishra

20.1 Resources for R

- dplyr and tidyr cheat sheet
- ggplot cheat sheet
- data table cheat sheet
- RMarkdown cheat sheet
- Hadley Wickham's R Style Guide
- Jade's R-for-epi course
- Tidy Eval in 5 Minutes (video)
- Tidy Evaluation (e-book)
- Data Frame Columns as Arguments to Dplyr Functions (blog)
- Standard Evaluation for *_join (stackoverflow)
- Programming with dplyr (package vignette)

20.2 Resources for Git & Github

- Data Camp introduction to Git
- Introduction to Github

20.3 Scientific figures

- Ten Simple Rules for Better Figures

20.4 Writing

- Tips on how to write a great science paper
- ICMJE Definition of authorship
- Nature article on elements of style for scientific writing
- The Pathway to Publishing: A Guide to Quantitative Writing in the Health Sciences
- Secret, actionable writing tips

20.5 Presentations

- How to tell a compelling story in scientific presentations
- How to give a killer narratively-driven scientific talk
- How to make a better poster
- How to make an even better poster

20.6 Professional advice

- Professional advice, especially for your first job

20.7 Funding

- Building Your Funding Train

20.8 Ethics and global health research

- Global Code of Conduct For Research in Resource-Poor Settings
- Who is a global health expert? Advice for aspiring global health experts
- Transforming Global Health Partnerships